

# The Complete Data Engineering Fundamentals Handbook

From Beginner to Industry Ready

Python • SQL • Databases • Git • ETL/ELT • Data Warehousing • Data Formats •  
Google Cloud Platform

---

┆ *"Data is the new oil, but Data Engineers are the refineries."*

## A Professional Onboarding Handbook for Junior Data Engineers

This handbook is designed to take a software engineer with basic programming knowledge and turn them into a job-ready Data Engineer. It mirrors the internal onboarding material used by modern technology companies to train new Data Engineering hires — covering programming, databases, pipelines, warehousing, cloud infrastructure, and career readiness.

---

## How to Use This Handbook

- Read chapters in order — each chapter builds on the last.
  - Type out every code example yourself. Do not copy-paste.
  - Attempt every exercise before checking a solution.
  - Keep a personal notes file as you go; you will refer back to it during real projects.
  - Treat the "Common Mistakes" boxes as a checklist before submitting any code review.
-

## Table of Contents

1. Introduction to Data Engineering
  2. Python for Data Engineering
  3. SQL
  4. Database Fundamentals
  5. Git
  6. ETL and ELT
  7. Data Warehousing
  8. Data Formats
  9. Google Cloud Platform
  10. BigQuery
  11. Complete End-to-End Data Engineering Project
  12. Industry Best Practices
  13. Career Preparation
- 

<div style="page-break-after: always;"></div>

## CHAPTER 1: Introduction to Data Engineering

### 1.1 Learning Objectives

By the end of this chapter, you will be able to:

- Define Data Engineering and explain why it exists as a discipline.
- Distinguish Data Engineering from Data Analytics, Data Science, ML Engineering, BI, and Software Engineering.
- Describe the full Data Engineering lifecycle from raw data to business value.

- Explain, at a high level, how data flows through a large streaming or e-commerce company.

## 1.2 Key Concepts

| Term          | Definition                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------|
| Data Engineer | An engineer who designs, builds, and maintains the systems that collect, store, and move data |
| Pipeline      | An automated sequence of steps that moves and transforms data from source to destination      |
| Data Source   | Any system that produces data (apps, sensors, third-party APIs, databases)                    |
| Data Sink     | The final destination where processed data is stored for use                                  |
| Data Consumer | Anyone or anything that uses the processed data — a dashboard, a model, a report              |

## 1.3 What Is Data Engineering?

Every modern company runs on data. A streaming service needs to know which shows people watch. A retailer needs to know which products sell. A ride-sharing app needs to know where cars are right now. None of this data arrives ready-to-use — it arrives messy, scattered across many systems, in many formats, and often at massive volume.

**Data Engineering is the discipline of building the systems that turn raw, messy data into clean, reliable, and accessible data that other people and systems can use.**

Think of a city's water supply. Rain falls (raw data), but nobody drinks rainwater directly. Water treatment plants and pipe networks (data pipelines) collect it, clean it, and deliver it reliably to every home (data consumers). The people who design and maintain that

treatment plant and pipe network are the Data Engineers of the water system. If the pipes leak, or the water is not treated properly, everyone downstream suffers — this is exactly what happens when data pipelines are poorly engineered.

A Data Engineer's job is **not** to analyze the water, or decide whether the water tastes good, or invent new filtration chemistry. Their job is to make sure clean water reliably arrives, at the right pressure, in the right place, on time, every time.

## 1.4 Why Companies Need Data Engineers

Without Data Engineers, a company's data is:

- **Scattered** — spread across production databases, third-party APIs, log files, and spreadsheets.
- **Inconsistent** — one system calls a customer's identifier `user_id`, another calls it `cust_no`.
- **Untrustworthy** — duplicate records, missing values, and silent failures corrupt reports.
- **Inaccessible at scale** — a query that works fine on 10,000 rows can take hours or crash on 10 billion rows.

Data Engineers solve this by building pipelines and platforms so that:

- Data Analysts can write a SQL query and trust the numbers.
- Data Scientists can pull a clean training dataset without weeks of manual cleanup.
- Business dashboards refresh automatically and accurately every morning.
- Machine Learning models get fresh, correctly-formatted features in production.

## 1.5 Data Engineering vs. Related Roles

A very common beginner mistake is confusing these roles. The table below defines each clearly.

| Role                                | Core Question They Answer                                    | Primary Tools                                      | Typical Output                                             |
|-------------------------------------|--------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------|
| <b>Data Engineering</b>             | "How do we reliably move and store data at scale?"           | Python, SQL, Airflow, Spark, cloud data warehouses | Pipelines, data warehouses, clean datasets, infrastructure |
| <b>Data Analytics</b>               | "What happened, and why?"                                    | SQL, Excel, Tableau, Looker                        | Dashboards, reports, insights                              |
| <b>Data Science</b>                 | "What pattern exists in this data, and what can we predict?" | Python, statistics, ML libraries                   | Models, experiments, statistical findings                  |
| <b>Machine Learning Engineering</b> | "How do we deploy and scale ML models in production?"        | Python, ML frameworks, Kubernetes, MLOps tools     | Deployed, monitored ML systems                             |
| <b>Business Intelligence (BI)</b>   | "How do we present metrics to decision-makers?"              | SQL, BI tools (Power BI, Looker, Tableau)          | Executive dashboards, KPI reports                          |
| <b>Software Engineering</b>         | "How do we build the application itself?"                    | Any general-purpose language, frameworks           | Web apps, backend services, mobile apps                    |

**Analogy:** Imagine a restaurant.

- The **Software Engineer** builds the restaurant (the building, the kitchen equipment, the ordering app).

- The **Data Engineer** builds the supply chain — trucks that deliver fresh ingredients (data) from farms (sources) to the kitchen (warehouse) reliably and on schedule.
- The **Data Analyst** is the person checking which dishes sold best last week.
- The **Data Scientist** is the chef experimenting with a new recipe based on customer taste patterns.
- The **ML Engineer** puts that new recipe into permanent rotation on the menu, at scale, across every branch.
- **BI** is the printed report the restaurant owner reads every morning summarizing sales.

## 1.6 The Complete Data Engineering Lifecycle

Every data pipeline, no matter how complex, follows the same fundamental lifecycle:



▼

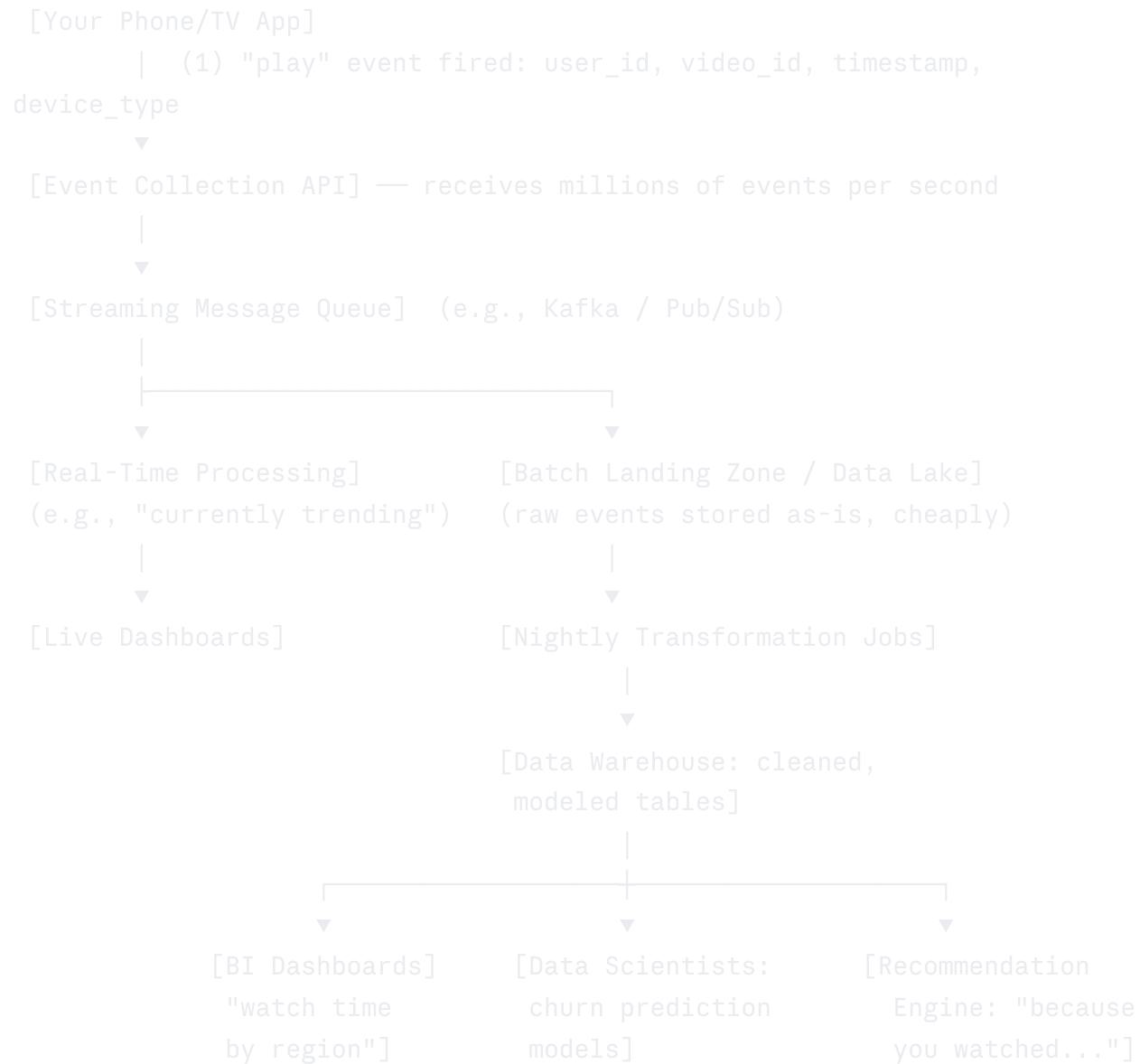


1. **Generate** — Data is created: a user clicks a button, a sensor reports temperature, a payment is processed.
2. **Ingest** — The data is captured and moved out of the source system, often via APIs, log collectors, or database change streams.
3. **Store** — The raw data lands somewhere durable — a data lake or a staging area — before it's touched.

4. **Transform** — The data is cleaned, validated, joined, and reshaped into a usable structure (this is the "T" in ETL/ELT, covered in Chapter 6).
5. **Serve** — The finished data is exposed to analysts, dashboards, applications, or ML models.
6. **Govern & Monitor** — Running invisibly across every stage: access control, data quality checks, lineage tracking, cost management, and alerting.

## 1.7 Real-World Example: How Data Flows at a Streaming Company

Let's trace what happens when you press "play" on a show at a company like Netflix, YouTube, or a fictional streaming service we'll call **Castify**.



Every one of the "boxes" above is built and maintained by Data Engineers:

- The event collection API and message queue are built to survive massive, bursty traffic (a new episode dropping causes a spike).

- The data lake stores raw, unprocessed events cheaply, in case something needs to be reprocessed later.
- The nightly transformation jobs clean, deduplicate, and reshape data into warehouse tables analysts can query.
- The warehouse tables feed dashboards, machine learning training pipelines, and recommendation systems.

This same pattern — **capture** → **land raw** → **transform** → **warehouse** → **serve** — appears at Amazon (orders and shipments), Spotify (listens and skips), and Airbnb (searches and bookings), just with different data.

## 1.8 Mini Exercise

Sketch (on paper or in a text file) the data flow for one of the following, using the six lifecycle stages from section 1.6:

1. A food delivery app tracking "order placed → order delivered."
2. A fitness app tracking daily step counts from a wearable device.

## 1.9 Challenge Exercise

Pick a real company (any company you use daily). List three different data sources it likely has (e.g., mobile app events, payment transactions, customer support tickets), and describe in 2–3 sentences how those sources might need to be combined to answer one business question, such as "Which customers are at risk of leaving?"

## 1.10 Common Mistakes

⚠ **Common Mistake:** Believing a Data Engineer's job is to "analyze data and find insights." That is the Data Analyst/Scientist's job. A Data Engineer's success is measured by *reliability, correctness, and scalability* of the data infrastructure — not by the insights themselves.

⚠ **Common Mistake:** Assuming all data pipelines run in real time. In practice, the majority of pipelines in most companies still run on a batch schedule (hourly or nightly), because it's simpler and cheaper. Real-time pipelines are used only when the business truly needs low latency.

⚠ **Common Mistake:** Treating "the data lake" and "the data warehouse" as the same thing. They serve different purposes — this is explained fully in Chapter 7.

## 1.11 Best Practices

- Always assume incoming data will be messy, late, duplicated, or incomplete — design for it from day one.
- Keep a copy of raw, untransformed data forever (or as long as storage cost allows) so you can reprocess if a transformation bug is discovered later.
- Document what each data source means and where it comes from — this "data lineage" saves enormous time during incident debugging.

## 1.12 Interview Questions

1. **Q: What is the difference between a Data Engineer and a Data Analyst?** A: A Data Engineer builds and maintains the infrastructure and pipelines that collect, store, and transform data reliably at scale. A Data Analyst consumes that data to answer business questions using queries, dashboards, and reports. Data Engineers focus on *how data moves and is structured*; Analysts focus on *what the data means for the business*.
2. **Q: Describe the Data Engineering lifecycle in your own words.** A: A strong answer walks through generation, ingestion, storage, transformation, serving, and governance, using a concrete example (see section 1.7).
3. **Q: Why might a company choose a nightly batch pipeline instead of real-time streaming?** A: Batch pipelines are simpler to build, easier to debug, and cheaper to run. Real-time streaming adds significant infrastructure complexity and cost, and

should only be used when the business genuinely needs low-latency data (e.g., fraud detection, live dashboards).

### 1.13 Review Questions

1. In your own words, what problem does Data Engineering solve for a company?
2. Name the six stages of the Data Engineering lifecycle in order.
3. Give one real-world example of raw data becoming a business decision.
4. Why is Software Engineering not the same as Data Engineering, even though both involve writing code?

### 1.14 Chapter Summary

Data Engineering exists because raw data is messy, scattered, and untrustworthy — someone must build reliable systems to collect, store, transform, and serve it. Data Engineers are distinct from Analysts, Data Scientists, ML Engineers, and BI professionals, though all roles work with the same underlying data. Every pipeline, regardless of company or industry, follows the same lifecycle: generate, ingest, store, transform, serve, and govern. Real companies like Netflix, Amazon, and Spotify all apply this same pattern to their own unique data.

### 1.15 Further Reading

- *Fundamentals of Data Engineering* by Joe Reis and Matt Housley (O'Reilly)
- *Designing Data-Intensive Applications* by Martin Kleppmann
- Google Cloud's "Data Engineering on Google Cloud" learning path ([cloud.google.com/training](https://cloud.google.com/training))

<div style="page-break-after: always;"></div>

# CHAPTER 2: Python for Data Engineering

## 2.1 Learning Objectives

By the end of this chapter, you will be able to:

- Use Python's core data types and control structures confidently.
- Read, write, and manipulate CSV and JSON files.
- Handle errors gracefully so pipelines fail safely instead of silently.
- Use virtual environments and pip to manage dependencies like a professional.
- Use `pandas` and `numpy` for the kind of data manipulation Data Engineers do daily.

## 2.2 Key Concepts

| Term                | Definition                                                     |
|---------------------|----------------------------------------------------------------|
| Variable            | A named reference to a value stored in memory                  |
| Data Type           | The category of value a variable holds (e.g., integer, string) |
| Function            | A reusable, named block of code                                |
| Module              | A single Python file containing reusable code                  |
| Package             | A folder of modules, installable via pip                       |
| Virtual Environment | An isolated Python installation for a single project           |
| Exception           | An error that occurs during program execution                  |

## 2.3 Why Python for Data Engineering?

Python is the dominant language in Data Engineering because it is readable, has a massive ecosystem of data libraries (`pandas`, `numpy`, `pyarrow`), integrates natively with cloud SDKs (Google Cloud, AWS, Azure), and is the language most orchestration tools (like Apache Airflow) are written in and configured with.

## 2.4 Variables and Data Types

A variable is a label attached to a value in memory.

```
python

row_count = 1523          # int
average_price = 19.99     # float
table_name = "orders"     # str
is_valid = True           # bool
```

Python is **dynamically typed** — you don't declare a type; Python infers it. But it is still important to know your types, because a pipeline that expects a number and receives text will crash.

```
python

print(type(row_count))    # <class 'int'>
print(type(average_price)) # <class 'float'>
print(type(table_name))   # <class 'str'>
```

## 2.5 Strings

Strings represent text — the most common data type you'll clean in Data Engineering (customer names, addresses, log lines).

```
python
```

```
name = "  Jane Doe  "
clean_name = name.strip().title()      # "Jane Doe"

email = "JANE@EXAMPLE.COM"
clean_email = email.lower()            # "jane@example.com"

log_line = "2026-07-07,ERROR,Connection timeout"
parts = log_line.split(",")            # ['2026-07-07', 'ERROR', 'Connection

message = f"Loaded {row_count} rows from {table_name}"
```

## Common string operations for Data Engineers:

| Method                                        | Purpose                            |
|-----------------------------------------------|------------------------------------|
| <code>.strip()</code>                         | Remove leading/trailing whitespace |
| <code>.lower()</code> / <code>.upper()</code> | Normalize case for comparison      |
| <code>.split(delim)</code>                    | Break a line into fields           |
| <code>.join(list)</code>                      | Combine a list into one string     |
| <code>.replace(a, b)</code>                   | Fix bad characters                 |
| f-strings                                     | Build messages/queries dynamically |

## 2.6 Numbers

python

```
total_orders = 4200          # int
tax_rate = 0.075             # float
revenue = total_orders * 25.50
rounded_revenue = round(revenue, 2)
```

Watch for **integer division vs. float division**:

```
python

print(7 / 2)    # 3.5  (float division)
print(7 // 2)   # 3    (integer/floor division)
print(7 % 2)    # 1    (remainder – useful for partitioning data into buckets)
```

## 2.7 Lists, Tuples, Sets, and Dictionaries

These four structures are the backbone of in-memory data manipulation before a library like pandas gets involved.

| Structure                      | Ordered?      | Mutable? | Duplicates Allowed? | Typical Use                                |
|--------------------------------|---------------|----------|---------------------|--------------------------------------------|
| List <code>[]</code>           | Yes           | Yes      | Yes                 | A sequence of rows or values               |
| Tuple <code>()</code>          | Yes           | No       | Yes                 | A fixed record, e.g., one (lat, long) pair |
| Set <code>{}</code>            | No            | Yes      | No                  | Deduplicating IDs                          |
| Dictionary <code>{k: v}</code> | Yes<br>(3.7+) | Yes      | Keys unique         | A single row as key-value pairs            |

```
python

# List: a batch of order IDs
order_ids = [1001, 1002, 1003, 1002]

# Tuple: a fixed coordinate
warehouse_location = (37.7749, -122.4194)

# Set: deduplicate customer IDs seen today
unique_customers = set([501, 502, 501, 503])
print(unique_customers)    # {501, 502, 503}

# Dictionary: a single row of data
order = {
    "order_id": 1001,
    "customer_id": 501,
    "amount": 49.99,
    "status": "shipped"
}
print(order["amount"])    # 49.99
```

A **list of dictionaries** is the most common way to represent a small table in raw Python:

```
python

orders = [
    {"order_id": 1001, "amount": 49.99},
    {"order_id": 1002, "amount": 19.99},
]
total = sum(o["amount"] for o in orders)
```

## 2.8 Functions

python

```
def clean_price(raw_price: str) -> float:
    """Convert a messy price string like '$19.99 ' into a float."""
    cleaned = raw_price.replace("$", "").strip()
    return float(cleaned)

print(clean_price("$19.99 "))    # 19.99
```

**Best practice:** always write small, single-purpose functions with type hints and docstrings — pipelines are easier to test and debug when logic is broken into named steps like `extract()`, `clean()`, `load()`.

## 2.9 Loops and Conditionals

python

```

raw_rows = ["1001,49.99,shipped", "1002,,pending", "1003,19.99,shipped"]

clean_rows = []
for row in raw_rows:
    fields = row.split(",")
    order_id, amount, status = fields
    if amount == "":
        continue # skip rows with missing amount
    clean_rows.append({
        "order_id": int(order_id),
        "amount": float(amount),
        "status": status
    })

print(clean_rows)

```

## 2.10 Modules and Packages

A **module** is a single `.py` file. A **package** is a folder of modules with an `__init__.py` (or, in modern Python, simply a folder with modules inside).

```

python

# file: cleaning.py
def clean_price(raw_price):
    return float(raw_price.replace("$", "").strip())

# file: main.py
from cleaning import clean_price
print(clean_price("$25.00"))

```

## 2.11 File Handling: CSV and JSON

## Reading and Writing CSV

python

```
import csv
```

```
# Reading
```

```
with open("orders.csv", newline="") as f:
```

```
    reader = csv.DictReader(f)
```

```
    for row in reader:
```

```
        print(row["order_id"], row["amount"])
```

```
# Writing
```

```
with open("clean_orders.csv", "w", newline="") as f:
```

```
    writer = csv.DictWriter(f, fieldnames=["order_id", "amount", "status"])
```

```
    writer.writeheader()
```

```
    writer.writerows(clean_rows)
```

## Reading and Writing JSON

python

```
import json

# Reading
with open("orders.json") as f:
    data = json.load(f)

# Writing
with open("clean_orders.json", "w") as f:
    json.dump(clean_rows, f, indent=2)

# Working with a JSON string directly
raw = '{"order_id": 1001, "amount": 49.99}'
order = json.loads(raw)
print(order["amount"])
```

**Always use the `with` statement** when opening files — it automatically closes the file, even if an error occurs.

## 2.12 Error Handling

Pipelines must **fail safely**. A single bad row should not crash an entire nightly job that processes millions of rows.

```
python
```

```
def parse_amount(raw_value: str) -> float:
    try:
        return float(raw_value)
    except ValueError:
        print(f"WARNING: could not parse amount '{raw_value}', defaulting to 0.0")
        return 0.0

def load_file(path: str):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        print(f"ERROR: file not found at {path}")
        raise
    finally:
        print("Attempted file load complete.")
```

| Exception         | Common Cause                                            |
|-------------------|---------------------------------------------------------|
| ValueError        | Converting text to a number fails                       |
| KeyError          | Accessing a dictionary key that doesn't exist           |
| FileNotFoundError | Reading a file/path that doesn't exist                  |
| TypeError         | Mixing incompatible types, e.g., <code>"5" + 5</code>   |
| ZeroDivisionError | Dividing by zero — common when computing rates/averages |

## 2.13 Virtual Environments and pip

A **virtual environment** isolates a project's dependencies so different projects can use different (even conflicting) library versions without interfering with each other.

```
bash

# Create a virtual environment
python -m venv venv

# Activate it (macOS/Linux)
source venv/bin/activate

# Activate it (Windows)
venv\Scripts\activate

# Install packages inside the isolated environment
pip install pandas numpy requests

# Save your dependencies for teammates
pip freeze > requirements.txt

# Recreate the same environment elsewhere
pip install -r requirements.txt
```

 **Common Mistake:** Installing packages globally (without a virtual environment). This causes "it works on my machine" bugs when a teammate or production server has different library versions.

## 2.14 Useful Libraries for Data Engineering

### `os` and `pathlib` — Working with Files and Paths

```
python

import os
from pathlib import Path

# os style
for filename in os.listdir("data/"):
    print(filename)

# pathlib style (modern, preferred)
data_dir = Path("data")
for file in data_dir.glob("*.csv"):
    print(file.name)
```

### `datetime` — Working with Dates and Times

```
python

from datetime import datetime, timedelta

now = datetime.now()
yesterday = now - timedelta(days=1)
formatted = now.strftime("%Y-%m-%d")    # "2026-07-07"
parsed = datetime.strptime("2026-07-07", "%Y-%m-%d")
```

### `requests` — Calling APIs

```
python
```

```
import requests

response = requests.get("https://api.example.com/orders", timeout=10)
if response.status_code == 200:
    data = response.json()
else:
    print(f"API call failed with status {response.status_code}")
```

## **pandas** — Tabular Data Manipulation

```
python

import pandas as pd

df = pd.read_csv("orders.csv")
print(df.head())
print(df.shape)                    # (rows, columns)
print(df.isnull().sum())           # count missing values per column

df["amount"] = df["amount"].fillna(0)
df_shipped = df[df["status"] == "shipped"]
summary = df.groupby("status")["amount"].sum()

df.to_csv("clean_orders.csv", index=False)
df.to_parquet("clean_orders.parquet")
```

## **numpy** — Numerical Computation

```
python
```

```
import numpy as np

amounts = np.array([49.99, 19.99, 0, 25.50])
print(amounts.mean())
print(amounts.std())
print(np.where(amounts == 0, np.nan, amounts)) # replace 0 with missing mark
```

## 2.15 Mini Project: Clean a Messy Orders File

Given a CSV with missing amounts, inconsistent casing in the `status` column, and duplicate `order_id`s, write a Python script that:

1. Reads the file using `pandas`.
2. Fills missing amounts with 0.
3. Normalizes `status` to lowercase.
4. Removes duplicate `order_id`s, keeping the first occurrence.
5. Writes the result to `clean_orders.csv`.

python

```
import pandas as pd

df = pd.read_csv("messy_orders.csv")
df["amount"] = df["amount"].fillna(0)
df["status"] = df["status"].str.lower().str.strip()
df = df.drop_duplicates(subset="order_id", keep="first")
df.to_csv("clean_orders.csv", index=False)
print(f"Cleaned {len(df)} rows.")
```

## 2.16 Exercises

1. Write a function `to_snake_case(column_name: str) -> str` that converts `"Customer Name"` into `"customer_name"`.
2. Read a JSON file containing a list of orders and print the total revenue.
3. Write a function that safely divides two numbers and returns `0` instead of raising `ZeroDivisionError`.

## 2.17 Challenge Exercise

Build a small script that:

- Reads a CSV of raw web server log lines (comma-separated: timestamp, status\_code, url).
- Counts how many requests returned each status code.
- Writes the summary to a JSON file as `{"200": 950, "404": 12, "500": 3}`.
- Handles any malformed lines without crashing (log a warning and skip them).

## 2.18 Common Mistakes

 **Common Mistake:** Using `==` to compare floating-point numbers. Due to floating-point precision, `0.1 + 0.2 == 0.3` is `False` in Python. Use `round()` or a small tolerance when comparing.

 **Common Mistake:** Mutating a list while iterating over it, which causes items to be skipped. Iterate over a copy (`for item in list(my_list):`) if you need to modify the original.

 **Common Mistake:** Catching a broad `except Exception:` and silently swallowing all errors. Always log what was caught, and only catch the specific exception you expect.

## 2.19 Best Practices

- Always use virtual environments per project.
- Pin dependency versions in `requirements.txt` for reproducibility.
- Write small, testable functions rather than one giant script.
- Log warnings for skipped/bad rows instead of silently dropping them — silent data loss is one of the most dangerous bugs in Data Engineering.
- Use type hints (`def clean(x: str) -> float:`) to make pipeline code self-documenting.

## 2.20 Interview Questions

1. **Q: What is the difference between a list and a tuple?** A: Lists are mutable (can be changed after creation) and typically hold a collection of similar items. Tuples are immutable and typically represent a fixed-size record, such as coordinates. Tuples are also hashable (when contents allow), so they can be used as dictionary keys.
2. **Q: How would you handle a malformed row in a 10-million-row CSV file without stopping the entire pipeline?** A: Wrap the parsing logic for each row in a try/except block, log the malformed row (including line number and reason) to a separate error log, and continue processing the remaining rows. This ensures one bad row doesn't halt the whole job.
3. **Q: Why use virtual environments?** A: They isolate a project's dependencies from the system Python and other projects, preventing version conflicts and ensuring the same code runs identically across development, testing, and production.

## 2.21 Review Questions

1. What's the difference between `/` and `//` in Python?
2. Name two ways to safely handle a `FileNotFoundError`.
3. Why would you choose a `set` over a `list` to store customer IDs?

4. What does `pip freeze > requirements.txt` do, and why is it useful?

## 2.22 Chapter Summary

Python is the primary language of Data Engineering because of its readability and rich ecosystem. Core data types (strings, numbers, lists, tuples, sets, dictionaries) let you represent and manipulate data before reaching for heavier tools. Functions, loops, and error handling let you build pipelines that process large volumes of messy data without crashing. Virtual environments and pip keep dependencies reproducible across machines. Libraries like `pandas` and `numpy` provide the vectorized, tabular operations Data Engineers use daily to clean, transform, and analyze data at scale.

## 2.23 Further Reading

- *Python for Data Analysis* by Wes McKinney (creator of pandas)
- Official Python documentation: [docs.python.org](https://docs.python.org)
- pandas documentation: [pandas.pydata.org](https://pandas.pydata.org)

<div style="page-break-after: always;"></div>

# CHAPTER 3: SQL

## 3.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain what a relational database is and how tables relate to each other.
- Write SELECT queries using filtering, aggregation, sorting, and joins.
- Use subqueries, CTEs, and window functions for advanced analysis.
- Understand transactions and why they matter.
- Recognize and apply basic query optimization techniques.

### 3.2 Key Concepts

| Term           | Definition                                                  |
|----------------|-------------------------------------------------------------|
| Table          | A structured collection of rows and columns                 |
| Row (Record)   | A single entry in a table                                   |
| Column (Field) | A named attribute shared by every row                       |
| Primary Key    | A column (or set of columns) uniquely identifying each row  |
| Foreign Key    | A column referencing the primary key of another table       |
| Index          | A data structure that speeds up lookups on a column         |
| Constraint     | A rule enforced on a column's data (e.g., NOT NULL, UNIQUE) |

### 3.3 What Is SQL and Why It Matters

**SQL (Structured Query Language)** is how humans and applications communicate with relational databases. Almost every Data Engineering task — extracting data, transforming it, loading it into a warehouse, validating it — eventually touches SQL. It is, without exaggeration, the single most important skill for a Data Engineer.

### 3.4 Databases, Tables, Rows, and Columns

A **database** is an organized collection of tables. A **table** looks like a spreadsheet:

Each row is one order. Each column holds one type of information across all rows.

### 3.5 Keys, Indexes, and Constraints

- **Primary Key:** uniquely identifies each row (e.g., `order_id`). No two rows can share one, and it cannot be NULL.
- **Foreign Key:** a column in one table that points to the primary key in another, creating a relationship. `orders.customer_id` referencing `customers.customer_id` means every order must belong to a real customer.
- **Index:** like a book's index — instead of scanning every page, the database jumps straight to matching rows. Indexes dramatically speed up `WHERE` and `JOIN` operations but slightly slow down writes (since the index must also be updated).
- **Constraints:** rules like `NOT NULL`, `UNIQUE`, `CHECK (amount >= 0)` that the database enforces automatically, preventing bad data from ever being inserted.

sql

```

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    signup_date DATE NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    amount DECIMAL(10,2) CHECK (amount >= 0),
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE INDEX idx_orders_customer ON orders(customer_id);

```

### 3.6 Normalization

**Normalization** is the process of organizing tables to reduce data duplication and prevent inconsistency. Instead of repeating a customer's email in every order row, we store customers once and reference them by ID.

| Normal Form | Rule (Simplified)                                                               |
|-------------|---------------------------------------------------------------------------------|
| 1NF         | Each column holds a single, atomic value (no comma-separated lists in one cell) |
| 2NF         | 1NF + every non-key column depends on the <i>whole</i> primary key              |
| 3NF         | 2NF + no non-key column depends on another non-key column                       |

**Trade-off:** Highly normalized designs minimize duplication but require more joins. Data warehouses often intentionally *denormalize* for read performance (see Chapter 7).

### 3.7 DDL vs. DML

| Category | Stands For                 | Commands                                                                              | Purpose                   |
|----------|----------------------------|---------------------------------------------------------------------------------------|---------------------------|
| DDL      | Data Definition Language   | <code>CREATE</code> , <code>ALTER</code> , <code>DROP</code>                          | Defines/changes structure |
| DML      | Data Manipulation Language | <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> | Reads/changes data        |

```
sql

-- DDL
ALTER TABLE orders ADD COLUMN discount_pct DECIMAL(5,2);
DROP TABLE IF EXISTS temp_orders;

-- DML
INSERT INTO orders (order_id, customer_id, amount, status)
VALUES (1004, 503, 75.00, 'pending');

UPDATE orders SET status = 'shipped' WHERE order_id = 1004;

DELETE FROM orders WHERE status = 'cancelled';
```

### 3.8 SELECT, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT

```
sql
```

```

-- Basic selection
SELECT order_id, amount, status FROM orders;

-- Filtering
SELECT * FROM orders WHERE status = 'shipped' AND amount > 20;

-- Aggregation
SELECT status, COUNT(*) AS order_count, SUM(amount) AS total_amount
FROM orders
GROUP BY status;

-- Filtering on aggregated results (HAVING, not WHERE)
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 100;

-- Sorting and limiting
SELECT * FROM orders
ORDER BY amount DESC
LIMIT 5;

```

**Key distinction:** `WHERE` filters rows *before* grouping. `HAVING` filters groups *after* aggregation. Using `WHERE` when you mean `HAVING` (e.g., `WHERE SUM(amount) > 100`) will raise an error in most databases.

### 3.9 JOINS

Joins combine rows from two or more tables based on a related column.

| INNER JOIN                                                                        | LEFT JOIN                                                                         | RIGHT JOIN                                                                          | FULL                                                                                |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| OUTER JOIN                                                                        |                                                                                   |                                                                                     |                                                                                     |
|  |  |  |  |
| only matching<br>everything from<br>rows in both<br>(NULL where<br>match)         | all of A + matches<br>from B (NULL if none)                                       | all of B + matches<br>from A (NULL if none)                                         | both<br>no                                                                          |

sql

```
-- INNER JOIN: only orders that have a matching customer
SELECT o.order_id, c.email, o.amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;

-- LEFT JOIN: all customers, even those with zero orders
SELECT c.customer_id, c.email, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

| Join Type       | Returns                                                             |
|-----------------|---------------------------------------------------------------------|
| INNER JOIN      | Only rows with matches in both tables                               |
| LEFT JOIN       | All rows from the left table, matched rows from the right (or NULL) |
| RIGHT JOIN      | All rows from the right table, matched rows from the left (or NULL) |
| FULL OUTER JOIN | All rows from both tables, matched where possible                   |

### 3.10 UNION

sql

```
-- Combine two result sets with the same columns, removing duplicates
SELECT customer_id FROM orders_2025
UNION
SELECT customer_id FROM orders_2026;

-- UNION ALL keeps duplicates (and is faster, since no dedup step is needed)
SELECT customer_id FROM orders_2025
UNION ALL
SELECT customer_id FROM orders_2026;
```

### 3.11 Subqueries

A subquery is a query nested inside another query.

sql

```
-- Find customers who spent more than the average order amount
SELECT customer_id, amount
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);

-- Subquery in the FROM clause
SELECT status, avg_amount
FROM (
    SELECT status, AVG(amount) AS avg_amount
    FROM orders
    GROUP BY status
) AS status_summary
WHERE avg_amount > 25;
```

### 3.12 Views

A **view** is a saved, reusable query that behaves like a virtual table.

```
sql

CREATE VIEW shipped_orders AS
SELECT * FROM orders WHERE status = 'shipped';

SELECT * FROM shipped_orders WHERE amount > 30;
```

### 3.13 Common Table Expressions (CTEs)

CTEs (**WITH** clauses) make complex queries readable by naming intermediate steps.

```
sql
```

```
WITH customer_totals AS (  
    SELECT customer_id, SUM(amount) AS total_spent  
    FROM orders  
    GROUP BY customer_id  
)  
  
high_value_customers AS (  
    SELECT * FROM customer_totals WHERE total_spent > 100  
)  
  
SELECT c.email, h.total_spent  
FROM high_value_customers h  
JOIN customers c ON h.customer_id = c.customer_id;
```

### 3.14 Window Functions

Window functions perform calculations across a set of rows *related to the current row*, without collapsing them into a single output row like `GROUP BY` does.

sql

```

-- Rank customers by spend within each region
SELECT
    customer_id,
    region,
    total_spent,
    RANK() OVER (PARTITION BY region ORDER BY total_spent DESC) AS spend_rank
FROM customer_totals;

-- Running total of daily revenue
SELECT
    order_date,
    daily_revenue,
    SUM(daily_revenue) OVER (ORDER BY order_date) AS running_total
FROM daily_revenue_summary;

```

| Function                                 | Purpose                                                  |
|------------------------------------------|----------------------------------------------------------|
| <code>ROW_NUMBER()</code>                | Assigns a unique sequential number per row               |
| <code>RANK()</code>                      | Assigns rank, with ties sharing a rank (gaps after ties) |
| <code>DENSE_RANK()</code>                | Like RANK but no gaps after ties                         |
| <code>LAG()</code> / <code>LEAD()</code> | Access a previous/next row's value                       |
| <code>SUM()/AVG() OVER()</code>          | Running totals/averages                                  |

### 3.15 Transactions

A **transaction** groups multiple statements so they all succeed or all fail together — critical when, for example, transferring money between two accounts.

sql

```
BEGIN TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
```

```
COMMIT;
```

```
-- if anything failed: ROLLBACK;
```

**ACID properties** (explained fully in Chapter 4) guarantee transactions behave predictably even under failures or concurrent access.

### 3.16 Query Optimization Tips

- Filter early: put `WHERE` conditions on indexed columns.
- Select only the columns you need — avoid `SELECT *` in production pipelines.
- Use `EXPLAIN` (or `EXPLAIN ANALYZE`) to see the query execution plan before optimizing blindly.
- Avoid functions on indexed columns in `WHERE` clauses (e.g., `WHERE YEAR(order_date) = 2026` prevents index use; prefer `WHERE order_date >= '2026-01-01'`).
- Prefer `EXISTS` over `IN` with large subqueries in many databases.
- Add indexes on columns frequently used in `JOIN` and `WHERE`, but don't over-index — every index slows down writes.

### 3.17 Mini Exercise

Write a query to find the top 3 customers by total amount spent, using the `orders` table.

### 3.18 Challenge Exercise

Using `orders` and `customers` tables, write a single query (using a CTE and a window function) that returns, for each customer, their most recent order date and their rank among all customers by total lifetime spend.

### 3.19 Common Mistakes

⚠ **Common Mistake:** Confusing `WHERE` and `HAVING`. Remember: `WHERE` filters rows before grouping; `HAVING` filters after.

⚠ **Common Mistake:** Forgetting `GROUP BY` includes every non-aggregated column in the `SELECT` list — omitting one causes an error (or unpredictable results) in most databases.

⚠ **Common Mistake:** Using `SELECT *` in production ETL code — it silently breaks when the source table's schema changes (new columns appear, or column order changes).

### 3.20 Best Practices

- Always write explicit column lists instead of `SELECT *`.
- Use meaningful table aliases (`o` for orders, `c` for customers) for readability.
- Format SQL consistently — one clause per line, keywords capitalized.
- Test queries on a small sample before running on a full production table.

### 3.21 Interview Questions

1. **Q: What is the difference between `WHERE` and `HAVING`?** A: `WHERE` filters individual rows before any grouping occurs. `HAVING` filters *groups* after aggregation (e.g., after a `GROUP BY` and `SUM()`), because aggregate results don't exist yet at the `WHERE` stage.

2. **Q: Explain the difference between `INNER JOIN` and `LEFT JOIN`.** A: `INNER JOIN` returns only rows with matches in both tables. `LEFT JOIN` returns all rows from the left table regardless of a match, filling in NULLs for unmatched columns from the right table.
3. **Q: What is a window function, and how is it different from `GROUP BY`?** A: A window function computes a value across a set of related rows (a "window") without collapsing them into one row per group, unlike `GROUP BY`, which reduces each group to a single summary row. This lets you show, for example, a running total alongside every individual row.

### 3.22 Review Questions

1. What's the difference between a primary key and a foreign key?
2. Why does normalization reduce data duplication?
3. When would you use a CTE instead of a subquery?
4. What does `RANK()` do differently from `ROW_NUMBER()`?

### 3.23 Chapter Summary

SQL is the universal language for interacting with relational data. Understanding tables, keys, and constraints lets you design reliable schemas. `SELECT`, `WHERE`, `GROUP BY`, and `JOIN` form the foundation of nearly every query you'll write. Subqueries, CTEs, and window functions unlock advanced analysis without sacrificing readability. Transactions ensure multi-step changes remain consistent, and thoughtful indexing and query design keep pipelines fast even at massive scale.

### 3.24 Further Reading

- *SQL Performance Explained* by Markus Winand
- Mode Analytics' free SQL tutorial ([mode.com/sql-tutorial](https://mode.com/sql-tutorial))
- PostgreSQL official documentation: [postgresql.org/docs](https://postgresql.org/docs)

```
<div style="page-break-after: always;"></div>
```

# CHAPTER 4: Database Fundamentals

## 4.1 Learning Objectives

- Distinguish relational (SQL) from non-relational (NoSQL) databases.
- Design a simple database schema using an ER diagram.
- Explain ACID properties and the CAP theorem.
- Choose the right database engine for a given use case.

## 4.2 Key Concepts

| Term                | Definition                                                                        |
|---------------------|-----------------------------------------------------------------------------------|
| Relational Database | Stores data in structured tables with enforced relationships                      |
| NoSQL Database      | Stores data in flexible, often non-tabular formats (documents, key-value, graphs) |
| ER Diagram          | A visual map of entities and their relationships                                  |
| ACID                | Guarantees for reliable transaction processing                                    |
| CAP Theorem         | A trade-off model for distributed databases                                       |

## 4.3 Relational Databases

Relational databases (MySQL, PostgreSQL, SQL Server) organize data into tables with strict schemas and enforce relationships via foreign keys. They excel at consistency and complex queries involving multiple related entities (e.g., orders, customers, products).

## 4.4 NoSQL Databases

NoSQL databases relax the strict tabular structure to gain flexibility and horizontal scalability.

| NoSQL Type  | Example             | Best For                                                  |
|-------------|---------------------|-----------------------------------------------------------|
| Document    | MongoDB             | Semi-structured data like user profiles, product catalogs |
| Key-Value   | Redis               | Caching, session storage, extremely fast lookups          |
| Wide-Column | Cassandra, Bigtable | Massive write volume, time-series data                    |
| Graph       | Neo4j               | Highly connected data — social networks, fraud detection  |

```
json

// A MongoDB-style document - flexible schema
{
  "customer_id": 501,
  "name": "Jane Doe",
  "addresses": [
    {"type": "home", "city": "Nairobi"},
    {"type": "work", "city": "Mombasa"}
  ],
  "preferences": {"newsletter": true}
}
```

## 4.5 Database Design and ER Diagrams

An **Entity-Relationship (ER) Diagram** visually maps out entities (tables) and how they relate.



This diagram shows: one customer has many orders; one order has many order items — a classic **one-to-many** relationship chain.

## 4.6 Relationship Types

| Relationship | Example                                       |
|--------------|-----------------------------------------------|
| One-to-One   | A person and their passport                   |
| One-to-Many  | A customer and their orders                   |
| Many-to-Many | Students and courses (needs a junction table) |

```
sql

-- Many-to-many requires a junction (bridge) table
CREATE TABLE student_courses (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

## 4.7 ACID Properties

| Property    | Meaning                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------|
| Atomicity   | A transaction fully happens or fully doesn't — no partial updates                             |
| Consistency | A transaction moves the database from one valid state to another, never violating constraints |
| Isolation   | Concurrent transactions don't interfere with each other's intermediate results                |

| Property   | Meaning                                                      |
|------------|--------------------------------------------------------------|
| Durability | Once committed, data survives even a crash immediately after |

**Example:** A bank transfer debiting one account and crediting another must be atomic — if the system crashes after the debit but before the credit, the transaction must roll back entirely, or money disappears.

## 4.8 The CAP Theorem

For a distributed database, the CAP theorem states you can only fully guarantee **two of three** properties at once:



- **Consistency:** every read receives the most recent write.
- **Availability:** every request receives a (non-error) response.
- **Partition Tolerance:** the system keeps working despite network failures between nodes.

Since network partitions *will* happen in any distributed system, real systems choose between prioritizing Consistency (CP, e.g., traditional relational clusters, HBase) or Availability (AP, e.g., Cassandra, DynamoDB) when a partition occurs.

## 4.9 Common Database Engines — When to Use Each

| Engine     | Type                  | Best For                                                                       |
|------------|-----------------------|--------------------------------------------------------------------------------|
| MySQL      | Relational            | Web applications, general-purpose OLTP                                         |
| PostgreSQL | Relational            | Complex queries, strong standards compliance, extensibility (JSON, geospatial) |
| SQLite     | Relational (embedded) | Local apps, prototypes, mobile apps, small single-file databases               |
| MongoDB    | NoSQL (Document)      | Flexible/evolving schemas, content management, catalogs                        |
| BigQuery   | Analytical (columnar) | Large-scale analytics, data warehousing (see Chapters 7 & 10)                  |

#### 4.10 Mini Exercise

Draw (in ASCII or on paper) an ER diagram for a small library system with `books`, `members`, and `loans` (a member borrows a book on a date).

#### 4.11 Challenge Exercise

You are asked to design a database for a ride-sharing app storing riders, drivers, and trips, where each trip has exactly one rider and one driver, but drivers and riders can have many trips. Design the schema (tables, primary keys, foreign keys) and explain your choice of relational vs. NoSQL for this use case.

#### 4.12 Common Mistakes

⚠ **Common Mistake:** Choosing MongoDB "because it's flexible" for data that is actually highly relational (e.g., financial transactions) — this leads to painful, error-prone application-level joins.

⚠ **Common Mistake:** Ignoring the CAP theorem trade-off and assuming a distributed NoSQL database is always both perfectly consistent and always available — in practice, during a network partition, it must sacrifice one.

#### 4.13 Best Practices

- Model relationships explicitly with foreign keys wherever the database supports it — don't rely on application code alone to maintain referential integrity.
- Choose NoSQL only when you have a specific reason (massive write scale, flexible/evolving schema, extreme low-latency key lookups) — default to relational for most business data.
- Always document your ER diagram alongside the schema so new engineers understand relationships quickly.

#### 4.14 Interview Questions

1. **Q: Explain the CAP theorem in your own words.** A: In a distributed database, when a network partition occurs, you must choose between consistency (always returning the latest data) and availability (always responding, even if the data might be slightly stale). You cannot guarantee both simultaneously during a partition.
2. **Q: When would you choose MongoDB over PostgreSQL?** A: When the data's structure varies significantly between records (e.g., product catalogs with wildly different attributes per category), or you need to scale writes horizontally across many servers more easily than a traditional relational engine allows.
3. **Q: What does "atomicity" mean in ACID?** A: A transaction either completes entirely or has no effect at all — there is no partial completion, even if a failure occurs mid-transaction.

## 4.15 Review Questions

1. Name three types of NoSQL databases and one use case for each.
2. What problem does a junction table solve?
3. Why can't a distributed system guarantee full consistency and full availability at the same time during a network partition?
4. When would SQLite be a better choice than PostgreSQL?

## 4.16 Chapter Summary

Relational databases enforce strict, related, tabular structure and excel at consistency and complex querying. NoSQL databases trade some of that structure for flexibility and horizontal scale. ER diagrams communicate schema design visually. ACID properties guarantee transaction reliability, while the CAP theorem describes the fundamental trade-offs distributed systems face. Choosing the right engine — MySQL, PostgreSQL, SQLite, MongoDB, or BigQuery — depends entirely on the shape of your data and your scale/consistency needs.

## 4.17 Further Reading

- *Designing Data-Intensive Applications* by Martin Kleppmann (essential for understanding distributed data systems)
- PostgreSQL vs. MongoDB comparison articles on each project's official site
- "CAP Theorem, 12 Years Later" — Eric Brewer's retrospective paper

<div style="page-break-after: always;"></div>

# CHAPTER 5: Git

## 5.1 Learning Objectives

- Explain what Git is and why version control matters in Data Engineering.

- Perform the core daily Git workflow: clone, branch, commit, push, pull request.
- Resolve merge conflicts confidently.
- Follow professional Git best practices used on real engineering teams.

## 5.2 Key Concepts

| Term              | Definition                                                      |
|-------------------|-----------------------------------------------------------------|
| Repository        | A folder tracked by Git, containing your project's full history |
| Commit            | A saved snapshot of changes, with a message                     |
| Branch            | An independent line of development                              |
| Merge             | Combining changes from one branch into another                  |
| Rebase            | Replaying commits on top of another branch's history            |
| Clone             | Downloading a copy of a remote repository                       |
| Fork              | A personal copy of someone else's repository on GitHub          |
| Pull Request (PR) | A request to merge your branch into another, with review        |

## 5.3 Why Data Engineers Need Git

Data pipelines are code — SQL scripts, Python transformation logic, Airflow DAGs, dbt models. Like any software, this code changes constantly, is worked on by multiple engineers, and must be safely deployable. Git lets teams track every change, collaborate without overwriting each other's work, and roll back instantly if a pipeline change breaks production.

## 5.4 Repositories and Commits

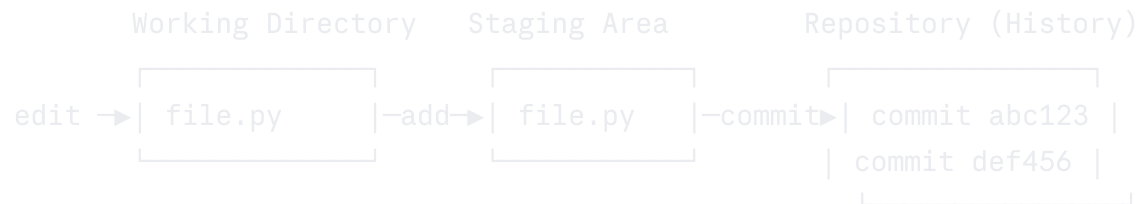
```
bash

# Start tracking a new project
git init

# Stage changes (mark them to be included in the next commit)
git add transform_orders.py

# Save a snapshot with a message
git commit -m "Add order amount cleaning logic"

# View history
git log --oneline
```



## 5.5 Branches

A branch lets you work on a feature or fix without touching the main, stable codebase.

```
bash
```

```
# Create and switch to a new branch
git checkout -b feature/clean-order-amounts

# ...make changes, commit them...

# Switch back to main
git checkout main
```

```
main:      A — B — C ————— F (merged)
              \               /
feature branch:  D — E —————
```

## 5.6 Merge and Rebase

```
bash

# Merge: combines feature branch into main, preserving both histories
git checkout main
git merge feature/clean-order-amounts

# Rebase: replays your branch's commits on top of the latest main (cleaner, 1
git checkout feature/clean-order-amounts
git rebase main
```

| Approach | History Shape                                | Use When                                                |
|----------|----------------------------------------------|---------------------------------------------------------|
| Merge    | Preserves branch structure (non-linear)      | Working on a shared branch, want full history preserved |
| Rebase   | Linear, as if changes were made sequentially | Cleaning up your own feature branch before opening a PR |

⚠ **Never rebase a branch that others are actively working on** — rebasing rewrites commit history, which breaks collaborators' copies of that branch.

## 5.7 Clone and Fork

```
bash

# Clone: download a copy of a repo you have access to
git clone https://github.com/company/data-pipelines.git
```

A **fork** (GitHub feature, not a Git command) creates your own copy of someone else's repository under your account — typically used for contributing to open-source projects you don't have direct write access to.

## 5.8 The GitHub Workflow (Pull Requests)

1. `git checkout -b feature/my-change`
2. Make changes, `git add`, `git commit`
3. `git push origin feature/my-change`
4. Open a Pull Request on GitHub
5. Teammates review, request changes, or approve
6. PR is merged into main
7. Branch is deleted

```
bash
```

```
git push origin feature/clean-order-amounts
```

## 5.9 Merge Conflicts

A conflict happens when two branches change the same lines of the same file differently. Git marks the conflicting sections:

```
<<<<<<< HEAD
amount = round(raw_amount, 2)
=====
amount = float(raw_amount)
>>>>>>> feature/clean-order-amounts
```

To resolve: manually edit the file to keep the correct code, remove the `<<<<<<<`, `=====`, `>>>>>>>` markers, then:

```
bash
```

```
git add transform_orders.py
git commit -m "Resolve merge conflict: keep rounding logic"
```

## 5.10 Daily Workflow and Best Practices

```
bash
```

```
# Start of day: get the latest changes
git checkout main
git pull origin main

# Create a focused branch for your task
git checkout -b fix/handle-null-amounts

# Work, commit often with clear messages
git add .
git commit -m "Handle null amounts by defaulting to 0"

# Push and open a PR
git push origin fix/handle-null-amounts
```

### Best practices:

- Write clear, imperative commit messages: `"Fix null handling in amount column"`, not `"fixed stuff"`.
- Keep commits small and focused on one logical change.
- Pull the latest `main` before starting new work to avoid painful conflicts later.
- Never commit secrets (API keys, passwords) — use `.gitignore` and environment variables instead.
- Review your own diff (`git diff`) before committing.

### 5.11 Mini Exercise

Create a local Git repository, make three commits representing three logical stages of cleaning a dataset, then view the history with `git log --oneline`.

## 5.12 Challenge Exercise

Simulate a merge conflict: create two branches from the same file, edit the same line differently in each, merge one into the other, and resolve the resulting conflict manually.

## 5.13 Common Mistakes

⚠ **Common Mistake:** Committing directly to `main` instead of working in a feature branch — this bypasses code review and risks breaking production pipelines.

⚠ **Common Mistake:** Writing vague commit messages like `"update"` or `"fix"` — six months later, nobody (including you) will know what changed or why.

⚠ **Common Mistake:** Committing large data files or secrets into Git history. Once committed, they remain in history even after deletion unless history is rewritten — use `.gitignore` from day one.

## 5.14 Best Practices

- One logical change per commit; one focused purpose per branch.
- Always open a Pull Request for review, even on personal projects — it builds the habit.
- Use `.gitignore` to exclude virtual environments, credentials, and large data files.
- Rebase your own local feature branches to keep history clean; never rebase shared branches.

## 5.15 Interview Questions

1. **Q: What's the difference between merge and rebase?** A: Merge combines two branches by creating a new commit that ties their histories together, preserving the branch structure. Rebase rewrites your branch's commits so they appear to have been made on top of the latest target branch, producing a linear history — but it should only be done on your own, unshared branches.

2. **Q: How do you resolve a merge conflict?** A: Git marks the conflicting sections in the file with `<<<<<<`, `=====`, and `>>>>>>`. You manually edit the file to keep the correct combination of changes, remove the conflict markers, then `git add` and `git commit` to finalize the resolution.
3. **Q: Why shouldn't you commit directly to the main branch on a team project?** A: Direct commits bypass code review, increasing the risk of introducing bugs or breaking changes into the shared, deployable codebase. Feature branches and pull requests let teammates review changes before they reach main.

## 5.16 Review Questions

1. What is the difference between `git clone` and `git fork`?
2. Why should feature branches be small and focused?
3. What happens if you rebase a branch that a teammate is also working on?
4. Why is `.gitignore` important in a Data Engineering project?

## 5.17 Chapter Summary

Git is essential infrastructure for any Data Engineering team, since pipelines are code that must be tracked, reviewed, and safely deployed. Repositories, commits, and branches form the foundation of version control. Merging and rebasing offer different trade-offs for combining work. The GitHub Pull Request workflow enables safe, reviewed collaboration, and understanding merge conflicts is essential for working on a shared codebase without fear.

## 5.18 Further Reading

- *Pro Git* by Scott Chacon and Ben Straub (free at [git-scm.com/book](https://git-scm.com/book))
- GitHub's official Git Handbook ([guides.github.com](https://guides.github.com))

# CHAPTER 6: ETL and ELT

## 6.1 Learning Objectives

- Define Extraction, Transformation, and Loading.
- Explain the difference between ETL and ELT and when each is preferred.
- Understand the role of workflow orchestration tools like Apache Airflow.
- Understand what dbt is and why it's central to the "Modern Data Stack."
- Build a simple pipeline that extracts, transforms, and loads CSV data.

## 6.2 Key Concepts

| Term           | Definition                                                              |
|----------------|-------------------------------------------------------------------------|
| Extraction     | Pulling data out of a source system                                     |
| Transformation | Cleaning, reshaping, validating, and enriching data                     |
| Loading        | Writing data into its final destination                                 |
| Pipeline       | An automated, ordered sequence of extract/transform/load steps          |
| Orchestration  | Scheduling, sequencing, and monitoring pipeline tasks                   |
| DAG            | Directed Acyclic Graph — a set of tasks with dependencies but no cycles |

## 6.3 Extraction

Extraction pulls raw data out of a source — a production database, a third-party API, flat files, or event streams.

python

```
import requests
import pandas as pd

# Extraction from an API
response = requests.get("https://api.example.com/orders", timeout=10)
raw_orders = pd.DataFrame(response.json())

# Extraction from a CSV export
raw_orders_csv = pd.read_csv("daily_orders_export.csv")
```

## 6.4 Transformation

Transformation is where raw data becomes trustworthy and usable: cleaning nulls, fixing types, deduplicating, joining reference data, and applying business logic.

```
python

# Transformation
df = raw_orders.copy()
df["amount"] = df["amount"].fillna(0)
df["status"] = df["status"].str.lower().str.strip()
df = df.drop_duplicates(subset="order_id")
df["order_date"] = pd.to_datetime(df["order_date"])
df["is_high_value"] = df["amount"] > 100
```

## 6.5 Loading

Loading writes the transformed data into its destination — a data warehouse table, a data lake file, or another database.

```
python
```

```
# Loading into a warehouse (conceptually – actual client varies by warehouse)
df.to_sql("orders_clean", con=engine, if_exists="append", index=False)

# Loading into a data lake as Parquet (efficient, compressed, columnar)
df.to_parquet("gs://my-bucket/orders/clean/orders_2026_07_07.parquet")
```

## 6.6 ETL vs. ELT

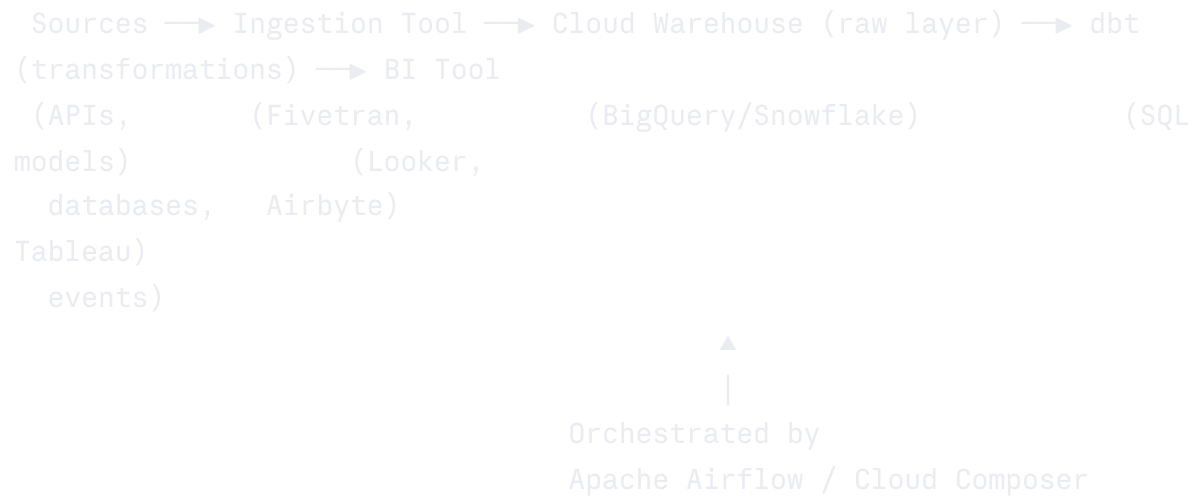
ETL: Extract → Transform (in a separate processing engine) → Load (into warehouse)

ELT: Extract → Load (raw data straight into warehouse) → Transform (inside the warehouse, using its own compute)

| Aspect                       | ETL                                                              | ELT                                                                |
|------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------|
| Where transformation happens | External processing engine, before loading                       | Inside the destination warehouse itself                            |
| Best suited for              | Legacy systems, smaller data volumes, strict pre-load validation | Modern cloud warehouses (BigQuery, Snowflake) with elastic compute |
| Raw data preserved?          | Often not, since transformation happens before storage           | Yes — raw data lands first, transformations are reproducible       |
| Typical tools                | Informatica, legacy SSIS pipelines                               | dbt, BigQuery SQL, Snowflake SQL                                   |

**Why ELT has become dominant:** modern cloud data warehouses (BigQuery, Snowflake, Redshift) can cheaply store raw data and scale compute on demand, so it's often simpler and more flexible to load first, then transform using the warehouse's own powerful SQL engine — and you always retain the original raw data for reprocessing.

## 6.7 The Modern Data Stack



This "Modern Data Stack" pattern separates concerns: ingestion tools handle extraction/loading, dbt handles transformation as pure SQL, and an orchestrator schedules and monitors everything.

## 6.8 Data Pipelines and Workflow Orchestration

A pipeline rarely runs as one script — it's usually a set of dependent tasks: "extract orders" must finish before "transform orders," which must finish before "load orders." **Orchestration tools** manage this scheduling, retrying, and alerting.

## 6.9 Apache Airflow

Airflow represents a pipeline as a **DAG** (Directed Acyclic Graph) — tasks with dependencies, but no circular loops.

python

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract():
    print("Extracting orders...")

def transform():
    print("Transforming orders...")

def load():
    print("Loading orders...")

with DAG(
    dag_id="daily_orders_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule_interval="@daily",
    catchup=False
) as dag:

    extract_task = PythonOperator(task_id="extract", python_callable=extract)
    transform_task = PythonOperator(task_id="transform", python_callable=transform)
    load_task = PythonOperator(task_id="load", python_callable=load)

    extract_task >> transform_task >> load_task # defines the dependency on
```

extract\_task → transform\_task → load\_task

Airflow provides scheduling, automatic retries on failure, a UI to monitor every run, and alerting when tasks fail.

## 6.10 dbt (data build tool)

**dbt** lets Data Engineers and Analysts write transformations as plain SQL (`SELECT` statements), and dbt handles turning them into tables/views, managing dependencies between them, and testing data quality.

```
sql

-- models/stg_orders.sql
SELECT
    order_id,
    customer_id,
    CAST(amount AS NUMERIC) AS amount,
    LOWER(TRIM(status)) AS status
FROM {{ source('raw', 'orders') }}
WHERE order_id IS NOT NULL
```

```
yaml

# models/schema.yml - dbt data quality tests
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: amount
        tests:
          - not_null
```

dbt automatically figures out the order to build models in, based on `{{ ref() }}` and `{{ source() }}` references between them — effectively building its own DAG from your SQL.

## 6.11 Cloud ETL Services

Cloud providers offer managed ETL/ELT tools so teams don't have to run their own orchestration servers:

| Provider     | Service        | Purpose                                           |
|--------------|----------------|---------------------------------------------------|
| Google Cloud | Dataflow       | Managed Apache Beam pipelines (batch + streaming) |
| Google Cloud | Cloud Composer | Managed Apache Airflow                            |
| AWS          | Glue           | Managed ETL with a built-in data catalog          |
| Azure        | Data Factory   | Managed pipeline orchestration                    |

## 6.12 Complete Example Pipeline (CSV → Clean CSV)

```
python
```

```

import pandas as pd
from pathlib import Path

def extract(path: str) -> pd.DataFrame:
    return pd.read_csv(path)

def transform(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["amount"] = pd.to_numeric(df["amount"], errors="coerce").fillna(0)
    df["status"] = df["status"].str.lower().str.strip()
    df = df.drop_duplicates(subset="order_id")
    return df

def load(df: pd.DataFrame, output_path: str) -> None:
    df.to_csv(output_path, index=False)
    print(f"Loaded {len(df)} rows to {output_path}")

if __name__ == "__main__":
    raw = extract("raw_orders.csv")
    clean = transform(raw)
    load(clean, "clean_orders.csv")

```

### 6.13 Mini Exercise

Extend the pipeline above to log (print) how many rows had an invalid `amount` that was coerced to 0.

### 6.14 Challenge Exercise

Design (as an ASCII DAG) a pipeline for a food delivery company that: extracts orders from an API and restaurant menus from a CSV, joins them, calculates total revenue per

restaurant, and loads results into a warehouse table — including a data quality check step before the final load.

## 6.15 Common Mistakes

- ⚠ **Common Mistake:** Discarding raw data immediately after transforming it. If a transformation bug is found weeks later, you cannot reprocess without raw data.
- ⚠ **Common Mistake:** Building transformation logic with no automated tests (e.g., no check that `order_id` is always unique) — bad data silently flows downstream into dashboards and models.
- ⚠ **Common Mistake:** Treating ETL/ELT pipelines as "run once and forget." Source systems change schemas over time; pipelines need monitoring and alerting to catch breakage early.

## 6.16 Best Practices

- Always preserve raw/landing data separately from transformed data.
- Make transformations idempotent — running the same job twice should produce the same result, not duplicate data.
- Add data quality tests (uniqueness, not-null, referential integrity) at every transformation stage.
- Use an orchestrator (Airflow, Cloud Composer) rather than cron jobs once you have more than a couple of dependent tasks.

## 6.17 Interview Questions

1. **Q: What is the difference between ETL and ELT?** A: In ETL, data is transformed by an external processing engine before being loaded into the destination. In ELT, raw data is loaded directly into the destination (usually a cloud data warehouse), and transformation happens afterward using the warehouse's own compute — which preserves raw data and leverages elastic cloud compute.

2. **Q: What is a DAG, and why does Airflow use it?** A: A DAG (Directed Acyclic Graph) represents tasks and their dependencies without circular loops. Airflow uses DAGs to define pipeline task order, ensuring tasks run only after their dependencies succeed, while preventing impossible circular dependencies.
3. **Q: Why has ELT become more popular than traditional ETL in recent years?** A: Cloud data warehouses now offer cheap storage and elastic, on-demand compute, making it practical to load raw data first and transform it later using the warehouse's own SQL engine — simplifying architecture and preserving raw data for reprocessing.

## 6.18 Review Questions

1. Name the three stages of ETL and briefly describe each.
2. Why is idempotency important in a data pipeline?
3. What role does dbt play in the Modern Data Stack?
4. Give one example of a managed cloud ETL/orchestration service.

## 6.19 Chapter Summary

ETL and ELT both move data from source to destination, differing in when and where transformation happens. Modern cloud warehouses have shifted the industry toward ELT, paired with tools like dbt for SQL-based transformation and Airflow (or Cloud Composer) for orchestration. A reliable pipeline preserves raw data, tests data quality at every stage, and is built to be safely re-run.

## 6.20 Further Reading

- Apache Airflow official documentation ([airflow.apache.org](https://airflow.apache.org))
- dbt Labs' "What, exactly, is dbt?" ([getdbt.com](https://getdbt.com))
- *Fundamentals of Data Engineering*, Chapters on ETL/ELT and orchestration

# CHAPTER 7: Data Warehousing

## 7.1 Learning Objectives

- Distinguish Data Warehouses, Data Lakes, and Lakehouses.
- Explain OLTP vs. OLAP workloads.
- Design fact and dimension tables using star and snowflake schemas.
- Understand partitioning and clustering for warehouse performance.

## 7.2 Key Concepts

| Term            | Definition                                                                           |
|-----------------|--------------------------------------------------------------------------------------|
| Data Warehouse  | A structured, query-optimized store for analytical data                              |
| Data Lake       | A store for raw data in any format, at any scale, at low cost                        |
| Lakehouse       | An architecture combining data lake storage with warehouse-like structure/querying   |
| OLTP            | Online Transaction Processing — many small, fast read/write operations               |
| OLAP            | Online Analytical Processing — complex queries over large volumes of historical data |
| Fact Table      | A table of measurable business events (e.g., sales)                                  |
| Dimension Table | A table of descriptive context (e.g., customer, product, time)                       |

### 7.3 Data Warehouse vs. Data Lake vs. Lakehouse

| DATA LAKE                                                                                                                                                                                                                                                                                    | DATA WAREHOUSE                                                                                                                                                                  | LAKEHOUSE                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <div> <div> <div>Raw, any format structured, (JSON, images, logs, file formats)</div> <div>Parquet, CSV...</div> <div>warehouse-like SQL</div> <div>Cheap, scalable engine on top</div> <div>storage (e.g. BigLake, Delta)</div> </div> <div>"store everything" of both worlds"</div> </div> | <div> <div>Structured, modeled tables, strict schema</div> <div>enforced</div> <div>Optimized for fast analytical queries</div> </div> <div>"query trusted, modeled data"</div> | <div> <div>Raw + open</div> <div>+</div> <div>(e.g.</div> </div> <div>"best"</div> |

| Aspect        | Data Lake                       | Data Warehouse                       | Lakehouse                            |
|---------------|---------------------------------|--------------------------------------|--------------------------------------|
| Schema        | Schema-on-read (flexible)       | Schema-on-write (strict)             | Both supported                       |
| Cost          | Very low                        | Higher (optimized compute + storage) | Low storage, warehouse-like querying |
| Typical Users | Data Engineers, Data Scientists | Analysts, BI tools                   | Both                                 |
| Example Tech  | Google Cloud Storage, AWS S3    | BigQuery, Snowflake, Redshift        | BigLake, Databricks Delta Lake       |

## 7.4 OLTP vs. OLAP

| Aspect        | OLTP                                                 | OLAP                                                                        |
|---------------|------------------------------------------------------|-----------------------------------------------------------------------------|
| Purpose       | Run the business day-to-day (e.g., process an order) | Analyze the business historically (e.g., total sales by region, by quarter) |
| Query pattern | Many small reads/writes, single-row focused          | Few large queries scanning millions of rows                                 |
| Example       | Your app's production database                       | A data warehouse table used for reporting                                   |
| Schema        | Highly normalized                                    | Often denormalized for read speed                                           |

Data Engineers build pipelines that move data *from* OLTP systems (where the business runs) *into* OLAP systems (where the business is analyzed) — this is precisely the ETL/ELT process from Chapter 6.

## 7.5 Fact Tables and Dimension Tables

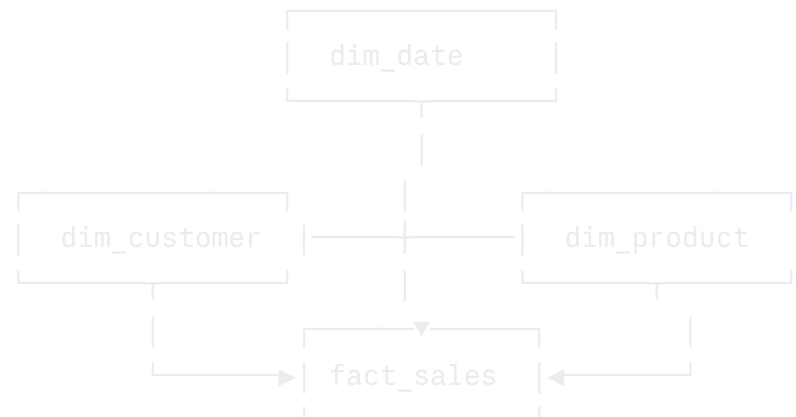
- **Fact table:** holds numeric, measurable events — one row per order, per click, per transaction.
- **Dimension table:** holds descriptive attributes used to slice and filter facts — customer details, product details, dates.

fact\_sales

|            |             |                   |                     |        |
|------------|-------------|-------------------|---------------------|--------|
| sale_id PK | date_key FK | product_key<br>FK | customer_<br>key FK | amount |
|------------|-------------|-------------------|---------------------|--------|

## 7.6 Star Schema

The **star schema** places one central fact table surrounded directly by its dimension tables — simple, denormalized, and fast to query.



sql

```

SELECT
    d.year,
    p.category,
    SUM(f.amount) AS total_sales
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY d.year, p.category;

```

## 7.7 Snowflake Schema

The **snowflake schema** further normalizes dimension tables into sub-dimensions, reducing redundancy at the cost of more joins.

```

dim_product → dim_category → dim_department
(normalized further into related, smaller tables)

```

| Aspect              | Star Schema                  | Snowflake Schema                                                                      |
|---------------------|------------------------------|---------------------------------------------------------------------------------------|
| Dimension structure | Denormalized (flat)          | Normalized (nested)                                                                   |
| Query complexity    | Simpler, fewer joins         | More joins required                                                                   |
| Storage             | Slightly more redundant      | Less redundant                                                                        |
| Query speed         | Generally faster             | Can be slower due to extra joins                                                      |
| When to use         | Most analytical/BI use cases | When dimension reuse/consistency across many facts matters more than query simplicity |

## 7.8 Partitioning

**Partitioning** splits a large table into smaller physical segments based on a column's value (commonly date), so queries only scan relevant partitions instead of the entire table.

```
sql

-- BigQuery example: partition by day using the order_date column
CREATE TABLE sales.fact_orders
PARTITION BY DATE(order_date)
AS SELECT * FROM staging.orders;

-- This query only scans the January 2026 partitions, not the whole table
SELECT SUM(amount)
FROM sales.fact_orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31';
```

fact\_orders table (partitioned by day)

|            |            |            |     |            |
|------------|------------|------------|-----|------------|
| 2026-01-01 | 2026-01-02 | 2026-01-03 | ... | 2026-07-07 |
|------------|------------|------------|-----|------------|

▲ query with WHERE order\_date = '2026-01-02'  
only scans this partition

## 7.9 Clustering

**Clustering** sorts data *within* partitions (or the whole table) by one or more columns, so queries filtering on those columns skip irrelevant blocks of data without needing a full partition per value.

sql

```
CREATE TABLE sales.fact_orders
PARTITION BY DATE(order_date)
CLUSTER BY customer_id, status
AS SELECT * FROM staging.orders;
```

| Aspect      | Partitioning                                                | Clustering                                      |
|-------------|-------------------------------------------------------------|-------------------------------------------------|
| Granularity | Coarse (e.g., by day)                                       | Fine (sorts within partitions)                  |
| Best for    | Columns with a manageable number of distinct values (dates) | High-cardinality columns (customer_id, user_id) |
| Combine?    | Yes — commonly used together                                | Yes                                             |

## 7.10 Real-World Warehouse Architecture Example

[OLTP production DBs] → [Raw/Landing Zone: Cloud Storage]

|



[Staging tables in BigQuery]

|

(dbt transformations)



[Modeled star schema: fact\_orders,  
dim\_customer, dim\_product, dim\_date]



## 7.11 Mini Exercise

Design a star schema for a movie streaming service tracking "watch events" — identify the fact table's measures and at least three dimension tables.

## 7.12 Challenge Exercise

Given a `fact_orders` table with 5 billion rows spanning 3 years, propose a partitioning and clustering strategy, and explain in 3-4 sentences how it improves both query cost and speed for a dashboard that always filters by `order_date` in the last 30 days and frequently filters by `customer_id`.

## 7.13 Common Mistakes

- ⚠ **Common Mistake:** Over-normalizing a data warehouse (snowflake schema everywhere) when a simpler star schema would serve BI queries faster with acceptable redundancy.
- ⚠ **Common Mistake:** Forgetting to partition large fact tables — queries end up scanning (and billing for) the entire table's history even when only a day's worth of data is needed.
- ⚠ **Common Mistake:** Confusing a data lake with a "dumping ground" — without cataloging and governance, a data lake becomes an unusable "data swamp."

## 7.14 Best Practices

- Partition large fact tables by date; cluster by high-cardinality filter columns.
- Prefer a star schema for most BI/analytical workloads — it's simpler and usually faster.

- Keep raw data in the lake even after warehouse loading, to support reprocessing and auditing.
- Document each dimension and fact table's grain (what one row represents) clearly.

## 7.15 Interview Questions

1. **Q: What is the difference between a data warehouse and a data lake?** A: A data warehouse stores structured, modeled, schema-enforced data optimized for fast analytical queries. A data lake stores raw data of any format at low cost, with schema applied only when read. Lakehouses combine both, adding warehouse-like querying on top of lake storage.
2. **Q: Explain the difference between a star schema and a snowflake schema.** A: A star schema has a central fact table joined directly to denormalized dimension tables, minimizing joins. A snowflake schema further normalizes those dimensions into related sub-tables, reducing redundancy but requiring more joins per query.
3. **Q: What's the difference between partitioning and clustering in a data warehouse?** A: Partitioning splits a table into physically separate segments based on a column (commonly date), so queries scan only relevant segments. Clustering sorts data within those segments by additional columns, letting queries skip irrelevant data blocks even for high-cardinality columns that wouldn't make sense to partition by directly.

## 7.16 Review Questions

1. What is the "grain" of a fact table, and why does it matter?
2. Why do OLAP systems often denormalize data, unlike OLTP systems?
3. Give one reason to choose a snowflake schema over a star schema.
4. How does partitioning reduce query cost in a cloud data warehouse?

### 7.17 Chapter Summary

Data warehouses provide fast, structured querying over modeled, trusted data, while data lakes provide cheap, flexible storage for raw data at any scale — with lakehouses aiming to combine both. OLTP systems run the business; OLAP systems analyze it, typically via star or snowflake schemas built from fact and dimension tables. Partitioning and clustering are essential performance techniques that keep large-scale warehouse queries fast and cost-efficient.

### 7.18 Further Reading

- *The Data Warehouse Toolkit* by Ralph Kimball (the definitive book on dimensional modeling)
- Google Cloud's BigQuery partitioning and clustering documentation
- Databricks' "What is a Lakehouse?" article

<div style="page-break-after: always;"></div>

## CHAPTER 8: Data Formats

### 8.1 Learning Objectives

- Compare CSV, JSON, Parquet, Avro, ORC, and XML.
- Understand row-based vs. columnar storage and why it matters for performance.
- Choose the right format for a given use case (storage, streaming, analytics).

### 8.2 Key Concepts

| Term             | Definition                                        |
|------------------|---------------------------------------------------|
| Row-based format | Stores each row's data together (e.g., CSV, Avro) |

| Term             | Definition                                                        |
|------------------|-------------------------------------------------------------------|
| Columnar format  | Stores each column's data together (e.g., Parquet, ORC)           |
| Schema evolution | The ability for a data format to handle changing fields over time |
| Compression      | Reducing file size to save storage and speed up I/O               |

## 8.3 Row-Based vs. Columnar Storage

|                                                                                          |                                                                                           |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <p>ROW-BASED STORAGE</p> <p>(good for writing full records, transactional workloads)</p> | <p>COLUMNAR STORAGE</p> <p>(good for analytical queries that scan only a few columns)</p> |
| Row 1: [id, name, amount, status]                                                        | Column "id": [1,2,3]                                                                      |
| Row 2: [id, name, amount, status]                                                        | Column "name": [a,b,c]                                                                    |
| Row 3: [id, name, amount, status]                                                        | Column "amount": [x,y,z]                                                                  |
|                                                                                          | Column "status": [p,q,r]                                                                  |

If a query only needs `SUM(amount)`, a columnar format reads *only* the amount column from disk — dramatically less I/O than a row-based format, which must read every column of every row.

## 8.4 CSV (Comma-Separated Values)

```
CSV

order_id,customer_id,amount,status
1001,501,49.99,shipped
1002,502,19.99,pending
```

| Aspect             | Detail                                                                |
|--------------------|-----------------------------------------------------------------------|
| Structure          | Row-based, plain text                                                 |
| Human-readable     | Yes                                                                   |
| Schema enforcement | None — types must be inferred or assumed                              |
| Compression        | Minimal natively; can be gzip-compressed externally                   |
| Best for           | Small datasets, quick exports, sharing with non-technical users       |
| Weaknesses         | No native types, no nested data, fragile with commas/quotes in values |

## 8.5 JSON (JavaScript Object Notation)

```
json
{
  "order_id": 1001,
  "customer_id": 501,
  "amount": 49.99,
  "status": "shipped",
  "items": [
    {"product_id": 55, "quantity": 2}
  ]
}
```

| Aspect             | Detail                                                                     |
|--------------------|----------------------------------------------------------------------------|
| Structure          | Row-based, supports nested/hierarchical data                               |
| Human-readable     | Yes                                                                        |
| Schema enforcement | None (flexible/schema-less)                                                |
| Best for           | APIs, semi-structured data, event payloads, config files                   |
| Weaknesses         | Verbose (repeated key names), slower to parse at scale than binary formats |

## 8.6 Parquet

**Parquet** is a columnar, binary format built for analytical workloads, widely used in data lakes and warehouses.

| Aspect             | Detail                                                              |
|--------------------|---------------------------------------------------------------------|
| Structure          | Columnar, binary                                                    |
| Human-readable     | No (binary)                                                         |
| Schema enforcement | Strong — schema stored in the file itself                           |
| Compression        | Excellent (column-level compression, often 5-10x smaller than CSV)  |
| Best for           | Data lakes, analytical queries, BigQuery/Spark pipelines            |
| Weaknesses         | Not human-readable; requires libraries (pyarrow, pandas) to inspect |

```
python

import pandas as pd
df = pd.read_csv("orders.csv")
df.to_parquet("orders.parquet", compression="snappy")

df2 = pd.read_parquet("orders.parquet")
```

## 8.7 Avro

**Avro** is a row-based, binary format popular in streaming systems (like Kafka), known for excellent **schema evolution** support.

| Aspect           | Detail                                                                      |
|------------------|-----------------------------------------------------------------------------|
| Structure        | Row-based, binary                                                           |
| Schema           | Stored alongside data (often in a schema registry)                          |
| Best for         | Streaming pipelines, systems where schemas change over time                 |
| Schema evolution | Excellent — can add/remove fields with backward/forward compatibility rules |

## 8.8 ORC (Optimized Row Columnar)

**ORC** is another columnar binary format, originally developed for the Hadoop ecosystem, often compared directly to Parquet.

| Aspect              | Detail                                                                                                                      |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Structure           | Columnar, binary                                                                                                            |
| Best for            | Hive/Hadoop-based analytical workloads                                                                                      |
| Compression         | Excellent, similar to Parquet                                                                                               |
| Compared to Parquet | Very similar in purpose; Parquet has broader adoption across the modern cloud ecosystem (BigQuery, Spark, cloud data lakes) |

## 8.9 XML (eXtensible Markup Language)

```
xml

<order>
  <order_id>1001</order_id>
  <customer_id>501</customer_id>
  <amount>49.99</amount>
</order>
```

| Aspect         | Detail                                                                                     |
|----------------|--------------------------------------------------------------------------------------------|
| Structure      | Row-based, tag-based, hierarchical                                                         |
| Human-readable | Yes, but verbose                                                                           |
| Best for       | Legacy enterprise systems, certain industry standards (finance, healthcare)                |
| Weaknesses     | Very verbose compared to JSON; slower to parse; largely superseded by JSON for new systems |

## 8.10 Comparison Table

| Format  | Type     | Human-Readable | Schema                | Compression | Best Use Case             |
|---------|----------|----------------|-----------------------|-------------|---------------------------|
| CSV     | Row      | Yes            | None                  | Poor        | Quick exports, small data |
| JSON    | Row      | Yes            | None (flexible)       | Poor        | APIs, nested/event data   |
| Parquet | Columnar | No             | Strong                | Excellent   | Data lakes, analytics     |
| Avro    | Row      | No             | Strong (evolving)     | Good        | Streaming (Kafka)         |
| ORC     | Columnar | No             | Strong                | Excellent   | Hadoop/Hive analytics     |
| XML     | Row      | Yes            | Schema (XSD) possible | Poor        | Legacy enterprise systems |

## 8.11 Performance Example

Loading and querying 1 GB of data with a single-column aggregation:

```
CSV:      Read entire 1 GB file → parse every column → aggregate (~slow)
Parquet:  Read only the "amount" column's compressed bytes (~fast, often 5-20x less I/O)
```

This is why cloud data warehouses like BigQuery are built around columnar storage internally, and why Parquet is the default recommended format for data lakes.

### 8.12 Mini Exercise

Convert a CSV file to Parquet using pandas, then compare the file sizes of both.

```
python

import pandas as pd
df = pd.read_csv("large_orders.csv")
df.to_parquet("large_orders.parquet")

import os
print("CSV size:", os.path.getsize("large_orders.csv"))
print("Parquet size:", os.path.getsize("large_orders.parquet"))
```

### 8.13 Challenge Exercise

You are designing a pipeline that ingests millions of events per hour from a mobile app via Kafka, lands them in a data lake, and later analyzes them in BigQuery. Recommend a format for (a) the Kafka streaming layer and (b) the data lake storage layer, and justify each choice in 2-3 sentences.

### 8.14 Common Mistakes

 **Common Mistake:** Storing large-scale analytical data as CSV or JSON in a data lake — this dramatically increases storage cost and query time compared to Parquet/ORC.

 **Common Mistake:** Choosing Avro for a data lake analytics layer when Parquet (columnar) would perform far better for read-heavy analytical queries — Avro shines in streaming/schema-evolution scenarios, not analytical scans.

⚠ **Common Mistake:** Assuming JSON has a "schema" just because individual documents look consistent — without an enforced schema, subtle inconsistencies (a field is sometimes a string, sometimes a number) can silently break downstream pipelines.

## 8.15 Best Practices

- Default to Parquet for analytical data lake storage.
- Use Avro (or Protobuf) for streaming pipelines where schema evolution matters.
- Reserve CSV/JSON for small-scale exports, human inspection, or system interoperability, not large-scale analytics.
- Always compress data at rest (Parquet/ORC compress natively; gzip CSV/JSON if you must use them at scale).

## 8.16 Interview Questions

1. **Q: Why is Parquet preferred over CSV for large-scale analytics?** A: Parquet is a columnar, compressed, binary format that lets queries read only the specific columns they need, dramatically reducing I/O and cost compared to CSV, which is row-based, uncompressed, and requires reading every column of every row.
2. **Q: When would you choose Avro over Parquet?** A: Avro is row-based and excels at streaming use cases (e.g., Kafka topics) where individual records are processed one at a time and schemas evolve frequently. Parquet is better suited to columnar analytical scans over large batches of data.
3. **Q: What is schema evolution, and why does it matter?** A: Schema evolution is a format's ability to gracefully handle changes to its structure over time (adding, removing, or renaming fields) without breaking existing consumers. It matters because production systems change constantly, and pipelines must not fail when a source schema changes in a backward-compatible way.

## 8.17 Review Questions

1. What is the core difference between row-based and columnar storage?
2. Name one strength and one weakness of JSON as a data format.
3. Why is Parquet the default choice for most modern data lakes?
4. In what scenario would XML still be encountered today?

## 8.18 Chapter Summary

Choosing the right data format has a direct impact on storage cost, query performance, and pipeline reliability. CSV and JSON are human-readable and flexible but inefficient at scale. Parquet and ORC are columnar, compressed, binary formats ideal for analytical workloads. Avro is a row-based format built for streaming and schema evolution. XML remains present mainly in legacy enterprise systems. Modern Data Engineering pipelines typically standardize on Parquet for storage and Avro (or Protobuf/JSON) for streaming.

## 8.19 Further Reading

- Apache Parquet official documentation ([parquet.apache.org](https://parquet.apache.org))
- Apache Avro official documentation ([avro.apache.org](https://avro.apache.org))
- "Columnar Storage" chapter in *Designing Data-Intensive Applications*

<div style="page-break-after: always;"></div>

# CHAPTER 9: Google Cloud Platform

## 9.1 Learning Objectives

- Explain core cloud computing concepts: regions, zones, projects, IAM, billing.
- Identify the purpose of key GCP services used in Data Engineering.
- Understand how these services fit together in a typical architecture.

## 9.2 Key Concepts

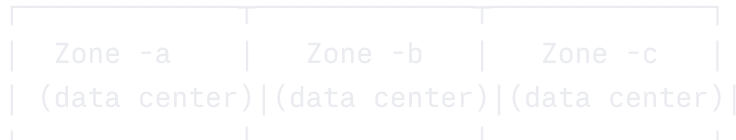
| Term    | Definition                                                                   |
|---------|------------------------------------------------------------------------------|
| Region  | A specific geographic location containing multiple data centers              |
| Zone    | An isolated location within a region                                         |
| Project | The base organizational unit for all GCP resources, billing, and permissions |
| IAM     | Identity and Access Management — controls who can do what on which resource  |

## 9.3 Cloud Computing Fundamentals

Cloud computing means renting computing resources (storage, processing power, databases) from a provider like Google, instead of buying and maintaining physical servers yourself. You pay for what you use, scale up or down instantly, and let the provider handle physical infrastructure, power, and networking.

## 9.4 Regions and Zones

Region: us-central1 (Iowa)

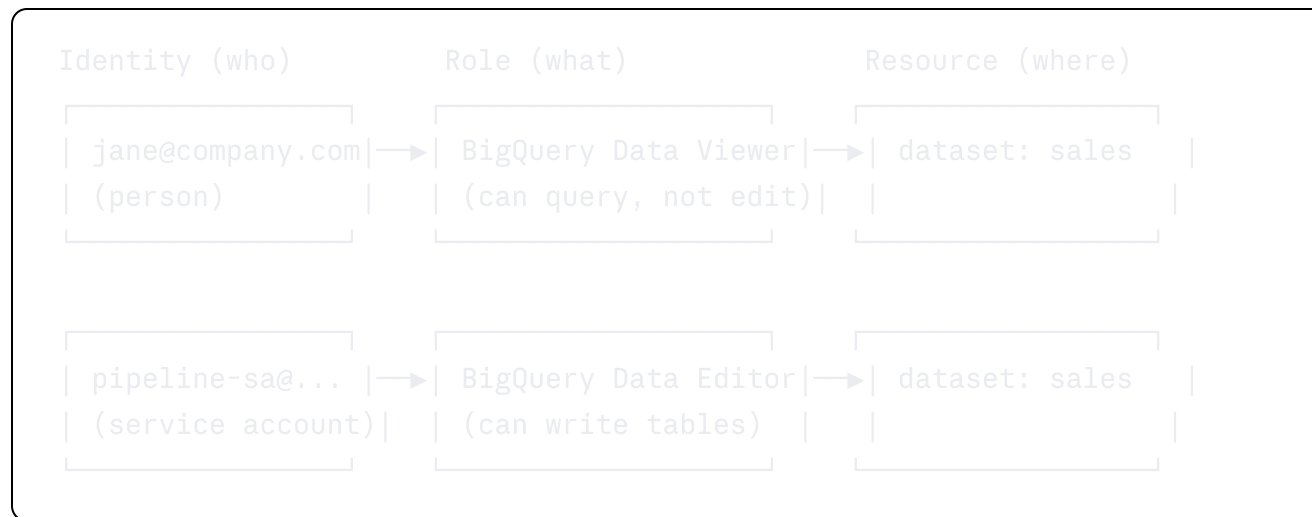


A **region** is a broad geographic area (e.g., `us-central1`). A **zone** is an isolated data center within that region (e.g., `us-central1-a`). Spreading resources across zones protects against a single data center failure; choosing a region close to your users reduces latency.

A **Project** is the top-level container in GCP — every resource (a BigQuery dataset, a storage bucket, a virtual machine) belongs to exactly one project. Projects have their own billing, IAM permissions, and APIs enabled.

## 9.6 IAM (Identity and Access Management)

IAM controls **who** (identity: a person or service account) can do **what** (role: a set of permissions) on **which resource**.



**Principle of least privilege:** grant only the minimum permissions needed for a task — never grant broad "Owner" or "Editor" roles for routine pipeline work.

## 9.7 Billing

GCP charges based on actual usage: storage consumed, compute time used, data queried (notably in BigQuery, billed per byte scanned unless using flat-rate pricing). Setting **budgets and alerts** is essential to avoid runaway costs from an inefficient query or forgotten running resource.

## 9.8 Core GCP Services for Data Engineers

| Service            | Category         | Purpose                                                              |
|--------------------|------------------|----------------------------------------------------------------------|
| Cloud Storage      | Storage          | Object storage for any file type — the foundation of most data lakes |
| Compute Engine     | Compute          | Virtual machines — full control over the OS and environment          |
| Cloud SQL          | Database         | Managed relational databases (MySQL, PostgreSQL, SQL Server)         |
| BigQuery           | Analytics        | Serverless, columnar data warehouse (see Chapter 10)                 |
| Pub/Sub            | Messaging        | Managed, scalable publish/subscribe messaging for streaming data     |
| Cloud Functions    | Compute          | Serverless, event-triggered code execution                           |
| Vertex AI          | Machine Learning | Managed platform for training and deploying ML models                |
| Dataflow           | Processing       | Managed Apache Beam for batch and streaming data processing          |
| Cloud Composer     | Orchestration    | Managed Apache Airflow                                               |
| Monitoring/Logging | Operations       | Observability into system health, errors, and performance            |

Cloud Storage holds files ("objects") in "buckets," forming the backbone of most data lakes.

```
bash

# Upload a file
gsutil cp orders.parquet gs://my-company-data-lake/orders/2026/07/07/

# List files
gsutil ls gs://my-company-data-lake/orders/2026/07/
```

| Storage Class | Use Case                                     |
|---------------|----------------------------------------------|
| Standard      | Frequently accessed data                     |
| Nearline      | Accessed roughly once a month                |
| Coldline      | Accessed roughly once a quarter              |
| Archive       | Long-term backup/compliance, rarely accessed |

## 9.10 Compute Engine

Virtual machines for workloads needing full control — e.g., a self-managed database, a custom application server, or heavy custom processing not suited to serverless options.

## 9.11 Cloud SQL

A fully managed relational database service — Google handles backups, patching, replication, and failover, so engineers focus on schema and queries instead of server

maintenance.

## 9.12 Pub/Sub

```
[Order Service] —publish→ [Pub/Sub Topic: "orders"] → [Subscriber 1:  
Fraud Check]  
                                                    ↳ [Subscriber  
2: Analytics Pipeline]  
                                                    ↳ [Subscriber  
3: Notification Service]
```

Pub/Sub decouples producers from consumers — the order service doesn't need to know or care who's listening; each subscriber processes events independently, at its own pace.

## 9.13 Cloud Functions

Small pieces of code that run automatically in response to an event — a new file landing in Cloud Storage, a Pub/Sub message arriving, an HTTP request — without managing any server.

```
python  
  
def process_new_file(event, context):  
    file_name = event['name']  
    bucket_name = event['bucket']  
    print(f"New file detected: gs://{bucket_name}/{file_name}")  
    # trigger downstream processing here
```

## 9.14 Vertex AI

Google's unified platform for the machine learning lifecycle: training, hyperparameter tuning, deployment, and monitoring of models — often the destination for data pipelines feeding ML feature sets.

## 9.15 Dataflow

A managed service for running Apache Beam pipelines that process data in batch or streaming mode, automatically scaling workers up or down based on volume.

## 9.16 Cloud Composer

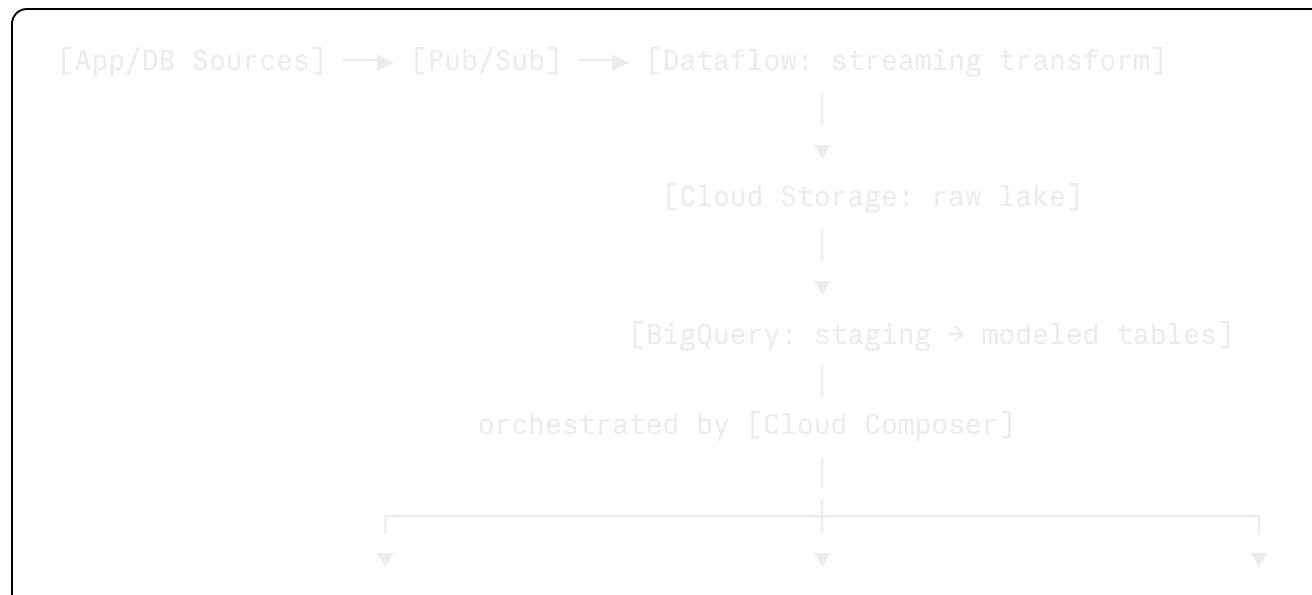
Google's managed Apache Airflow, removing the burden of hosting and scaling your own Airflow servers, while keeping the same DAG-based orchestration model from Chapter 6.

## 9.17 Monitoring and Logging

Every production pipeline needs observability:

- **Cloud Logging** captures logs from every service (e.g., a failed BigQuery load job).
- **Cloud Monitoring** tracks metrics (e.g., pipeline duration, error rate) and triggers alerts (e.g., "notify on-call if the nightly job hasn't succeeded by 6 AM").

## 9.18 A Typical GCP Data Architecture



## 9.19 Mini Exercise

Identify which GCP service you would use for each: (a) storing 50 TB of raw log files cheaply, (b) running a nightly Airflow-style pipeline, (c) receiving real-time clickstream events from a mobile app.

## 9.20 Challenge Exercise

Design a GCP architecture (as an ASCII diagram) for a company that receives IoT sensor readings every second from 10,000 devices, needs a live dashboard of current readings, and needs historical analysis going back 3 years. Identify which service handles each stage.

## 9.21 Common Mistakes

- ⚠ **Common Mistake:** Granting broad "Project Editor" or "Owner" IAM roles to service accounts used by pipelines, instead of narrowly scoped roles — a major security risk if credentials leak.
- ⚠ **Common Mistake:** Not setting billing budgets/alerts, leading to unexpectedly large bills from an inefficient BigQuery query scanning terabytes unnecessarily.
- ⚠ **Common Mistake:** Choosing Compute Engine (a full VM) for a task that a serverless option (Cloud Functions, Dataflow, BigQuery) would handle more simply and cheaply.

## 9.22 Best Practices

- Apply least-privilege IAM roles, especially for service accounts running pipelines.
- Use separate GCP projects for development, staging, and production environments.
- Tag/label resources for cost tracking by team or project.

- Prefer managed/serverless services (BigQuery, Dataflow, Cloud Functions) over self-managed VMs whenever possible, to reduce operational burden.

## 9.23 Interview Questions

1. **Q: What is the difference between a region and a zone in GCP?** A: A region is a broad geographic area containing multiple isolated data centers called zones. Distributing resources across zones within a region protects against a single data center outage, while choosing a nearby region reduces latency for end users.
2. **Q: What is the principle of least privilege, and how does it apply to IAM?** A: It means granting an identity only the minimum permissions necessary to perform its task — for example, giving a pipeline's service account "BigQuery Data Editor" on one specific dataset rather than "Owner" on the entire project, minimizing damage if credentials are ever compromised.
3. **Q: When would you use Pub/Sub in a data pipeline?** A: When you need to decouple data producers from consumers and reliably handle high-throughput, real-time event streams — for example, distributing order events to multiple independent downstream systems (fraud detection, analytics, notifications) without producers needing to know about consumers.

## 9.24 Review Questions

1. What does a GCP "project" contain, and why does every resource belong to one?
2. Name three GCP services and the specific Data Engineering purpose each serves.
3. Why is Cloud Composer preferred over self-hosting Airflow for most teams?
4. What's the difference between Cloud Storage classes like Standard and Coldline?

## 9.25 Chapter Summary

Google Cloud Platform provides the infrastructure building blocks for modern Data Engineering: Cloud Storage for data lakes, Cloud SQL and BigQuery for structured data,

Pub/Sub for streaming, Cloud Functions and Dataflow for processing, Vertex AI for machine learning, and Cloud Composer for orchestration. Understanding regions, zones, projects, IAM, and billing is essential before building any real pipeline on the platform — these are the guardrails that keep systems secure, reliable, and cost-controlled.

### 9.26 Further Reading

- Google Cloud's official documentation ([cloud.google.com/docs](https://cloud.google.com/docs))
- "Data Engineering on Google Cloud" learning path ([cloud.google.com/training](https://cloud.google.com/training))
- Google Cloud Architecture Center ([cloud.google.com/architecture](https://cloud.google.com/architecture))

<div style="page-break-after: always;"></div>

## CHAPTER 10: BigQuery

### 10.1 Learning Objectives

- Explain BigQuery's serverless, columnar architecture.
- Create datasets and tables, and load data into BigQuery.
- Write optimized SQL queries in BigQuery.
- Apply partitioning, clustering, and cost-control best practices.
- Understand BigQuery's pricing model and core security features.

### 10.2 Key Concepts

| Term    | Definition                                                       |
|---------|------------------------------------------------------------------|
| Dataset | A container for tables within a BigQuery project (like a schema) |
| Table   | A structured collection of rows, as in any relational system     |
| Slot    | A unit of BigQuery's computational capacity                      |

| Term              | Definition                                             |
|-------------------|--------------------------------------------------------|
| On-demand pricing | Pay per byte of data scanned by a query                |
| Flat-rate pricing | Pay a fixed price for dedicated query capacity (slots) |

### 10.3 BigQuery Architecture

BigQuery is a **serverless**, columnar data warehouse — you never provision servers or clusters. Google automatically manages storage and compute, scaling queries across thousands of machines as needed.



A key architectural feature: **storage and compute are separate**. You pay for storage independently from query compute, and BigQuery can scale compute elastically for a single large query without you managing any infrastructure.

### 10.4 Datasets and Tables

sql

```
-- Datasets are created via console/CLI, then referenced in SQL as project.dataset
CREATE SCHEMA IF NOT EXISTS `my-project.sales`;

CREATE TABLE `my-project.sales.orders` (
  order_id INT64,
  customer_id INT64,
  amount NUMERIC,
  status STRING,
  order_date DATE
);
```

## 10.5 Loading Data

bash

```
# Load a CSV file from Cloud Storage into a BigQuery table
bq load --source_format=CSV --skip_leading_rows=1 \
  my-project:sales.orders \
  gs://my-bucket/orders/orders.csv \
  order_id:INTEGER,customer_id:INTEGER,amount:NUMERIC,status:STRING,order_date:DATE
```

python

```
from google.cloud import bigquery

client = bigquery.Client()
job = client.load_table_from_uri(
    "gs://my-bucket/orders/orders.parquet",
    "my-project.sales.orders",
    job_config=bigquery.LoadJobConfig(source_format=bigquery.SourceFormat.PARQUET)
)
job.result()
print("Load complete.")
```

## 10.6 Writing SQL in BigQuery

BigQuery uses standard SQL with some extensions:

```
sql

-- Aggregation
SELECT status, COUNT(*) AS order_count, SUM(amount) AS total_revenue
FROM `my-project.sales.orders`
GROUP BY status;

-- Working with arrays and structs (BigQuery supports nested/repeated fields)
SELECT
    order_id,
    ARRAY_LENGTH(items) AS item_count
FROM `my-project.sales.orders_with_items`;
```

## 10.7 Optimization: Partitioning and Clustering

```
sql
```

```
CREATE TABLE `my-project.sales.fact_orders`  
PARTITION BY order_date  
CLUSTER BY customer_id  
AS SELECT * FROM `my-project.staging.orders`;
```

sql

```
-- GOOD: only scans partitions in the date range  
SELECT SUM(amount)  
FROM `my-project.sales.fact_orders`  
WHERE order_date BETWEEN '2026-07-01' AND '2026-07-07';  
  
-- BAD: wrapping the partition column in a function prevents partition pruning  
-- forcing BigQuery to scan the entire table  
SELECT SUM(amount)  
FROM `my-project.sales.fact_orders`  
WHERE EXTRACT(YEAR FROM order_date) = 2026;
```

## 10.8 Pricing

| Pricing Model              | How It Works                                                                     | Best For                          |
|----------------------------|----------------------------------------------------------------------------------|-----------------------------------|
| On-demand                  | Pay per byte scanned by each query (first 1 TB/month often free-tier)            | Variable, unpredictable workloads |
| Flat-rate / capacity-based | Pay for a fixed number of reserved slots regardless of usage                     | High, predictable query volume    |
| Storage                    | Billed separately per GB stored, with a discount for data untouched for 90+ days | All tables                        |

### Cost control tips:

- Always check the estimated bytes scanned before running a large query (BigQuery's UI shows this before execution).
- Avoid `(SELECT *)`; select only needed columns, since BigQuery bills by columns scanned, not rows.
- Use partitioning/clustering aggressively to reduce bytes scanned.

```
sql

-- Use a dry run to preview cost before running
-- (in bq CLI): bq query --dry_run --use_legacy_sql=false 'SELECT * FROM sale
```

## 10.9 Best Practices

- Partition large tables by date and cluster by frequently filtered columns.
- Avoid `(SELECT *)` — select only required columns to minimize bytes scanned and cost.
- Use scheduled queries or dbt for recurring transformations instead of manual runs.
- Set up cost controls: custom quotas, budget alerts, and query dry-runs before big jobs.
- Use materialized views for frequently repeated expensive aggregations.

## 10.10 Security

| Feature            | Purpose                                                    |
|--------------------|------------------------------------------------------------|
| IAM roles          | Control who can query, edit, or administer datasets/tables |
| Row-level security | Restrict which rows a user can see within a shared table   |

| Feature                              | Purpose                                                             |
|--------------------------------------|---------------------------------------------------------------------|
| Column-level security (data masking) | Hide or mask sensitive columns (e.g., SSNs) from unauthorized users |
| VPC Service Controls                 | Restrict data movement to/from approved network boundaries          |
| Encryption                           | Data is encrypted at rest and in transit by default                 |

```
sql
```

```
-- Row-level security example: restrict a sales rep to see only their region'
CREATE ROW ACCESS POLICY region_filter
ON `my-project.sales.fact_orders`
GRANT TO ("group:sales-us@company.com")
FILTER USING (region = 'US');
```

## 10.11 Real-World Example: Daily Sales Report Pipeline

```
[Raw CSVs in Cloud Storage]
  |
  ▼
[BigQuery Load Job: staging.orders_raw]
  |
  (dbt/SQL transformation)
  ▼
[BigQuery: sales.fact_orders – partitioned by order_date, clustered by
customer_id]
  |
  ▼
[Scheduled Query: aggregates yesterday's sales into sales.daily_summary]
```

↓  
[Looker Studio Dashboard – refreshed each morning]

## 10.12 Mini Exercise

Write a BigQuery SQL query that returns total revenue per day for the last 7 days from a partitioned `fact_orders` table, ensuring the query only scans relevant partitions.

## 10.13 Challenge Exercise

A team complains their BigQuery bill doubled last month. Given they run a `SELECT *` query against a 2 TB, non-partitioned table every 15 minutes, propose three specific changes to reduce cost, and estimate (conceptually) the relative impact of each.

## 10.14 Common Mistakes

⚠ **Common Mistake:** Using `SELECT *` on wide tables — BigQuery bills based on the columns scanned, so this can cost far more than selecting only the 2-3 needed columns.

⚠ **Common Mistake:** Filtering on a partition column wrapped in a function (e.g., `EXTRACT(YEAR FROM order_date) = 2026`), which disables partition pruning and forces a full table scan.

⚠ **Common Mistake:** Not setting IAM permissions carefully on datasets — over-permissive dataset access is a common source of data leaks.

## 10.15 Interview Questions

1. **Q: How does BigQuery's separation of storage and compute benefit users?** A: It allows storage costs and compute costs to scale independently — you can store huge volumes of data cheaply while only paying for compute when you actually

run queries, and BigQuery can allocate massive parallel compute to a single query without any manual cluster management.

2. **Q: Why does `SELECT *` cost more in BigQuery than selecting specific columns?**

A: Because BigQuery is a columnar store and bills (in on-demand pricing) based on the amount of data scanned per column, not per row. Selecting only necessary columns means BigQuery reads and bills for far less data.

3. **Q: How does partitioning improve BigQuery query performance and cost? A:**

Partitioning splits a table into date-based (or other) segments so a query filtering on the partition column only scans the relevant segments instead of the entire table, reducing both bytes scanned (cost) and query time.

## 10.16 Review Questions

1. What is the difference between on-demand and flat-rate BigQuery pricing?
2. Why is BigQuery described as "serverless"?
3. Name two security features BigQuery offers for protecting sensitive data.
4. Why does clustering complement partitioning rather than replace it?

## 10.17 Chapter Summary

BigQuery is Google Cloud's serverless, columnar data warehouse, separating storage and compute so it can scale queries elastically without manual infrastructure management. Effective use of BigQuery requires understanding its pricing model, writing column-efficient SQL, and applying partitioning and clustering to control cost and performance. Security features like IAM, row/column-level security, and encryption make BigQuery suitable for sensitive, enterprise-scale analytical workloads.

## 10.18 Further Reading

- Google's official BigQuery documentation ([cloud.google.com/bigquery/docs](https://cloud.google.com/bigquery/docs))
- "BigQuery: The Definitive Guide" by Valliappa Lakshmanan and Jordan Tigani

- Google Cloud's BigQuery best practices guide

<div style="page-break-after: always;"></div>

## **CHAPTER 11: Complete End-to-End Data Engineering Project**

### **11.1 Learning Objectives**

- Integrate everything from Chapters 1-10 into one working pipeline.
- Understand how raw CSV files become a queryable, visualized warehouse table in a real cloud environment.
- Practice explaining each stage of a pipeline the way you would in a design review or interview.

### **11.2 Project Overview**

**Scenario:** Castify, a fictional music streaming startup, receives daily CSV exports of song play events from its mobile app. We will build a pipeline that cleans this data, loads it into Google Cloud, models it in BigQuery, and visualizes it.

```
CSV files (raw play events)
|
▼
Python (extract + clean)
|
▼
Transformation (validate, dedupe, enrich)
|
▼
Upload to Google Cloud Storage (data lake, Parquet format)
|
▼
Load into BigQuery (staging table)
|
▼
Run SQL (transform into a star schema: fact_plays + dimensions)
|
▼
Visualize results (dashboard / notebook)
```

### 11.3 Step 1 — The Raw Data

csv

```
play_id,user_id,song_id,played_at,duration_seconds,device
1,501,9001,2026-07-07 08:15:00,182,mobile
2,502,9002,2026-07-07 08:16:30,,mobile
3,501,9001,2026-07-07 08:15:00,182,mobile
4,503,9003,2026-07-07 09:02:10,210,desktop
```

Notice the problems: row 2 is missing `duration_seconds`, and row 3 duplicates row 1 exactly.

## 11.4 Step 2 — Extract and Clean with Python

python

```
import pandas as pd

def extract(path: str) -> pd.DataFrame:
    return pd.read_csv(path)

def clean(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["played_at"] = pd.to_datetime(df["played_at"])
    df["duration_seconds"] = pd.to_numeric(df["duration_seconds"], errors="coerce")
    df["duration_seconds"] = df["duration_seconds"].fillna(df["duration_seconds"].mean())
    df = df.drop_duplicates(subset=["user_id", "song_id", "played_at"])
    df["device"] = df["device"].str.lower().str.strip()
    return df

raw = extract("play_events.csv")
clean_df = clean(raw)
print(f"Cleaned {len(clean_df)} of {len(raw)} raw rows")
```

## 11.5 Step 3 — Validate

python

```
def validate(df: pd.DataFrame) -> None:
    assert df["play_id"].is_unique, "Duplicate play_id detected!"
    assert df["duration_seconds"].min() >= 0, "Negative duration detected!"
    assert df["user_id"].notnull().all(), "Missing user_id detected!"
    print("Validation passed.")

validate(clean_df)
```

Validation acts as a safety gate — the pipeline should halt (or quarantine bad rows) rather than silently load broken data into the warehouse.

## 11.6 Step 4 — Upload to Google Cloud Storage as Parquet

python

```
clean_df.to_parquet("play_events_clean.parquet", compression="snappy")
```

bash

```
gsutil cp play_events_clean.parquet gs://castify-data-lake/plays/2026/07/07/
```

We chose **Parquet** (Chapter 8) because it's columnar, compressed, and the standard format for cloud data lakes feeding BigQuery.

## 11.7 Step 5 — Load into BigQuery

python

```
from google.cloud import bigquery

client = bigquery.Client()
job_config = bigquery.LoadJobConfig(source_format=bigquery.SourceFormat.PARQUET)

job = client.load_table_from_uri(
    "gs://castify-data-lake/plays/2026/07/07/play_events_clean.parquet",
    "castify.staging.play_events",
    job_config=job_config,
)
job.result()
print("Loaded into staging.play_events")
```

## 11.8 Step 6 — Model into a Star Schema with SQL

sql

```
-- Dimension: users
CREATE TABLE IF NOT EXISTS `castify.warehouse.dim_users` (
  user_id INT64,
  signup_date DATE
);

-- Dimension: songs
CREATE TABLE IF NOT EXISTS `castify.warehouse.dim_songs` (
  song_id INT64,
  title STRING,
  artist STRING
);

-- Fact table, partitioned and clustered per Chapter 7 & 10 best practices
CREATE TABLE IF NOT EXISTS `castify.warehouse.fact_plays`
PARTITION BY DATE(played_at)
CLUSTER BY user_id AS
SELECT
  play_id,
  user_id,
  song_id,
  played_at,
  duration_seconds,
  device
FROM `castify.staging.play_events`;
```

sql

```
-- Analytical query: total listening time per device type, last 7 days
SELECT
    device,
    SUM(duration_seconds) / 3600 AS total_hours_listened
FROM `castify.warehouse.fact_plays`
WHERE played_at >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY device
ORDER BY total_hours_listened DESC;
```

## 11.9 Step 7 — Orchestrate the Whole Pipeline with Airflow

python

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG(
    dag_id="castify_daily_plays_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule_interval="@daily",
    catchup=False,
) as dag:

    extract_clean = PythonOperator(task_id="extract_and_clean", python_callable=extract_and_clean)
    validate = PythonOperator(task_id="validate", python_callable=lambda: print("Validating data..."))
    upload_gcs = PythonOperator(task_id="upload_to_gcs", python_callable=lambda: print("Uploading to GCS..."))
    load_bq = PythonOperator(task_id="load_to_bigquery", python_callable=lambda: print("Loading to BigQuery..."))
    model_sql = PythonOperator(task_id="model_star_schema", python_callable=lambda: print("Modeling star schema..."))

    extract_clean >> validate >> upload_gcs >> load_bq >> model_sql
```

```
extract_and_clean → validate → upload_to_gcs → load_to_bigquery →  
model_star_schema
```

## 11.10 Step 8 — Visualize Results

The final `fact_plays` table (joined with its dimensions) is connected to a BI tool (e.g., Looker Studio) for a dashboard showing daily active listeners, top songs, and listening time by device — refreshed automatically each day as the Airflow DAG completes.

## 11.11 Explaining Every Step (Interview-Style Walkthrough)

| Step                    | What Happens                              | Why It Matters                                                |
|-------------------------|-------------------------------------------|---------------------------------------------------------------|
| Extract                 | Read raw CSV                              | Get data out of the source system                             |
| Clean                   | Fix types, fill missing values, dedupe    | Prevent garbage data downstream                               |
| Validate                | Assert data quality rules                 | Catch problems before they reach the warehouse                |
| Upload (Parquet to GCS) | Store compressed, columnar raw/clean data | Cheap, durable, fast to query later, reprocessable            |
| Load to BigQuery        | Move data into staging tables             | Make data queryable at scale                                  |
| Model (star schema)     | Build fact/dimension tables               | Optimize for fast, understandable analytical queries          |
| Orchestrate (Airflow)   | Schedule and sequence every step          | Ensure the pipeline runs reliably, daily, with retries/alerts |

| Step      | What Happens                 | Why It Matters                            |
|-----------|------------------------------|-------------------------------------------|
| Visualize | Connect BI tool to warehouse | Turn data into decisions for the business |

## 11.12 Mini Exercise

Add a data quality check to the pipeline that rejects (quarantines to a separate table) any row where `duration_seconds > 3600` (an implausibly long single "play").

## 11.13 Challenge Exercise

Extend this pipeline to also compute a `dim_date` table and a weekly "top 10 songs" materialized view, and describe (in prose or as a diagram) how you'd add a Slack alert if the Airflow DAG fails.

## 11.14 Common Mistakes

**⚠ Common Mistake:** Skipping the validation step "to save time" — this is precisely how bad data quietly corrupts dashboards and erodes stakeholder trust in the data team.

**⚠ Common Mistake:** Loading directly from CSV into BigQuery without first landing a clean, columnar copy in the data lake — this discards the reprocessing safety net discussed in Chapter 6.

**⚠ Common Mistake:** Building the star schema directly on top of messy staging data instead of validating and cleaning first — errors then propagate into every downstream fact/dimension table.

## 11.15 Best Practices

- Treat each pipeline stage (extract, clean, validate, load, model) as its own testable function or Airflow task.

- Always keep the clean, validated Parquet copy in the data lake — even after loading to BigQuery — as your reprocessing safety net.
- Add automated data quality checks (row counts, null checks, range checks) between every major stage.
- Document the pipeline's DAG and each table's grain so new team members can onboard quickly.

## 11.16 Interview Questions

1. **Q: Walk me through a data pipeline you've built, end to end.** A: A strong answer follows the extract → clean → validate → land raw → load → model → orchestrate → serve structure shown in this chapter, explaining the "why" behind each tool choice (e.g., "I used Parquet because it's columnar and compresses well for analytical queries").
2. **Q: Why land data in Cloud Storage before loading into BigQuery, instead of loading directly?** A: Landing in Cloud Storage preserves a cheap, durable, reprocessible raw/clean copy of the data, decoupled from the warehouse. If a downstream bug is found, the pipeline can be rerun from the lake without needing to re-extract from the original source.
3. **Q: How would you handle a pipeline failure at 2 AM before an executive dashboard refresh?** A: Describe monitoring/alerting via Airflow (task failure notifications), a runbook for common failure modes, and the value of idempotent tasks so a failed run can simply be retried without creating duplicate or corrupted data.

## 11.17 Review Questions

1. Why is the "validate" stage placed before loading into the data lake, not after?
2. What would happen if the pipeline used CSV instead of Parquet for the data lake layer?
3. Why does the fact table get partitioned by `played_at` specifically?

4. How does Airflow's task dependency ( $\boxed{>>}$ ) relate to the DAG concept from Chapter 6?

## 11.18 Chapter Summary

This chapter combined Python, cloud storage, BigQuery, SQL modeling, orchestration, and visualization into one coherent, realistic pipeline — mirroring what a junior Data Engineer builds and maintains in their first months on the job. Each stage exists for a specific reason: cleaning and validating protect data quality, landing data in Parquet preserves reprocessing ability, modeling into a star schema optimizes analytical queries, and orchestration ensures the whole system runs reliably without manual intervention.

## 11.19 Further Reading

- Google Cloud's "Data Engineering end-to-end" tutorials ([cloud.google.com/architecture](https://cloud.google.com/architecture))
- Apache Airflow's official "Best Practices" guide
- dbt Labs' example analytics projects on GitHub

<div style="page-break-after: always;"></div>

# CHAPTER 12: Industry Best Practices

## 12.1 Learning Objectives

- Apply professional software engineering discipline to Data Engineering work.
- Understand testing, automation, monitoring, and security as they apply to pipelines.
- Recognize what separates a junior engineer's code from a senior engineer's code.

## 12.2 Key Concepts

| Term          | Definition                                                                                   |
|---------------|----------------------------------------------------------------------------------------------|
| CI/CD         | Continuous Integration / Continuous Deployment — automated testing and release of code       |
| Observability | The ability to understand a system's internal state from its outputs (logs, metrics, traces) |
| Idempotency   | An operation that produces the same result no matter how many times it's run                 |
| Data Lineage  | A record of where data came from and how it was transformed                                  |

## 12.3 Version Control Discipline

Building on Chapter 5: every pipeline change — a new SQL model, a Python transformation fix, an Airflow DAG update — should go through a feature branch and Pull Request, with at least one reviewer, before reaching production.

## 12.4 Code Style

Consistent style makes code reviewable and maintainable across a team.

```
python
```

```
# GOOD: clear names, type hints, single responsibility
def clean_amount(raw_amount: str) -> float:
    return float(raw_amount.replace("$", "").strip())

# BAD: unclear names, no types, mixed responsibilities
def f(x):
    y = x.replace("$", "")
    return float(y.strip())
```

Use automated formatters (e.g., `black` for Python, `sqlfmt` for SQL) so style debates never happen in code review — the tool decides.

## 12.5 Documentation

- Every pipeline should have a short README: what it does, its schedule, its inputs/outputs, and who owns it.
- Every non-obvious transformation should have an inline comment explaining *why*, not just *what*.
- Data dictionaries should describe each table's columns and their meaning for downstream consumers.

## 12.6 Testing

| Test Type         | What It Checks                       | Example                                                                          |
|-------------------|--------------------------------------|----------------------------------------------------------------------------------|
| Unit test         | A single function's logic            | <code>clean_amount("\$19.99")</code> returns <code>19.99</code>                  |
| Data quality test | Properties of the data itself        | <code>order_id</code> column has no duplicates                                   |
| Integration test  | Multiple components working together | The full pipeline runs against a sample dataset and produces expected row counts |

```
python
```

```
def test_clean_amount():  
    assert clean_amount("$19.99 ") == 19.99  
    assert clean_amount("$0.00") == 0.0
```

```
sql
```

```
-- dbt-style data quality test (from Chapter 6)  
-- tests: unique, not_null on order_id
```

## 12.7 Automation

Manual pipeline runs don't scale and are error-prone. Automate:

- Scheduling (Airflow/Cloud Composer, not manual script runs).
- Testing (run tests automatically on every Pull Request via CI).
- Deployment (merging to main automatically deploys the updated DAG/dbt models).

## 12.8 Logging

```
python
```

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("orders_pipeline")

def load(df):
    logger.info(f>Loading {len(df)} rows into orders_clean")
    try:
        df.to_sql("orders_clean", con=engine, if_exists="append")
    except Exception as e:
        logger.error(f"Load failed: {e}")
        raise
```

Good logs answer: what happened, when, how much data was involved, and — if something failed — why.

## 12.9 Monitoring

Monitor pipelines the same way you'd monitor any production system:

- **Freshness:** did today's data actually arrive and load on time?
- **Volume:** is row count within an expected range (a sudden drop often signals an upstream problem)?
- **Duration:** is the pipeline taking noticeably longer than usual (a leading indicator of trouble)?
- **Failure alerts:** page or notify the on-call engineer immediately on failure, not the next morning.

## 12.10 Security

- Never hardcode credentials in code — use environment variables or a secrets manager.

- Apply least-privilege IAM (Chapter 9) to every pipeline's service account.
- Encrypt sensitive data at rest and in transit (most cloud services do this by default, but verify).
- Mask or tokenize sensitive fields (PII) in non-production environments.

```
python

import os
# GOOD
db_password = os.environ["DB_PASSWORD"]

# BAD — never do this
db_password = "SuperSecret123!"
```

## 12.11 Performance and Scalability

- Design pipelines to handle 10x today's data volume without a rewrite — partition, batch, and paginate where needed.
- Prefer set-based SQL operations over row-by-row processing in application code wherever possible; databases are optimized for set operations.
- Profile before optimizing — measure where the actual bottleneck is (I/O, computation, network) rather than guessing.

## 12.12 CI/CD for Data Pipelines

```
[Pull Request opened] → [CI runs: unit tests, SQL linting, dbt tests]
→ [Reviewer approves]
```

|

▼

```
[Merge to main]
|
▼
updated DAG/dbt models [CD: deploy
to
production automatically]
```

This ensures every change is tested and reviewed before reaching production, and deployment is repeatable and fast rather than manual and error-prone.

### 12.13 Mini Exercise

Take the `clean` function from Chapter 11's project and write two unit tests for it: one for missing `duration_seconds`, one for duplicate rows.

### 12.14 Challenge Exercise

Propose a monitoring plan (freshness, volume, duration, and alerting rules) for the Castify play-events pipeline from Chapter 11. Be specific about what values would indicate a problem.

### 12.15 Common Mistakes

**⚠ Common Mistake:** Treating testing as optional for "just a data pipeline" — untested transformation logic is one of the leading causes of silent, expensive data quality incidents.

**⚠ Common Mistake:** Logging too little ("Job finished") or too much (dumping entire DataFrames to logs) — both make debugging painful. Log meaningful, structured summaries.

**⚠ Common Mistake:** Hardcoding credentials or connection strings directly in scripts, which then get committed to Git — a serious, common security failure.

## 12.16 Best Practices

- Automated tests, CI/CD, and code review are not "extra" — they are standard professional practice for any pipeline touching production data.
- Design every pipeline task to be idempotent — safe to re-run without side effects like duplicate rows.
- Monitor freshness, volume, and duration for every critical pipeline, with alerts routed to a real person.
- Store secrets in a secrets manager or environment variables — never in code or version control.

## 12.17 Interview Questions

1. **Q: What does "idempotent" mean, and why does it matter for pipelines?** A: An idempotent operation produces the same result regardless of how many times it runs. It matters because pipelines fail and get retried; if a load step isn't idempotent, a retry after a partial failure could duplicate data instead of safely repeating the same result.
2. **Q: How would you monitor a daily data pipeline in production?** A: Track data freshness (did it arrive/complete on schedule), volume (is row count within expected bounds), and duration (is it running abnormally slow), with automated alerts routed to an on-call engineer the moment any of these deviate from expected ranges.
3. **Q: Why is CI/CD important for data pipelines, not just application code?** A: Pipelines are code, and errors in them can silently corrupt business-critical data. CI/CD ensures every change is automatically tested (unit tests, data quality tests) and reviewed before reaching production, catching bugs before they affect real data.

## 12.18 Review Questions

1. Name three types of tests relevant to a data pipeline.
2. Why should credentials never be hardcoded into pipeline scripts?
3. What three signals would you monitor to detect a pipeline going wrong?
4. What's the risk of a non-idempotent load step?

## 12.19 Chapter Summary

Professional Data Engineering applies the same rigor as professional software engineering: version control discipline, consistent code style, thorough documentation, automated testing, logging, monitoring, security, and CI/CD. These practices are what separate a fragile, manually-run script from a reliable, scalable, production-grade pipeline that a company can depend on every single day.

## 12.20 Further Reading

- *Fundamentals of Data Engineering*, Chapter on "Security" and "DataOps"
- Google's Site Reliability Engineering (SRE) book, free at [sre.google/books](https://sre.google/books)
- dbt Labs' testing and CI documentation

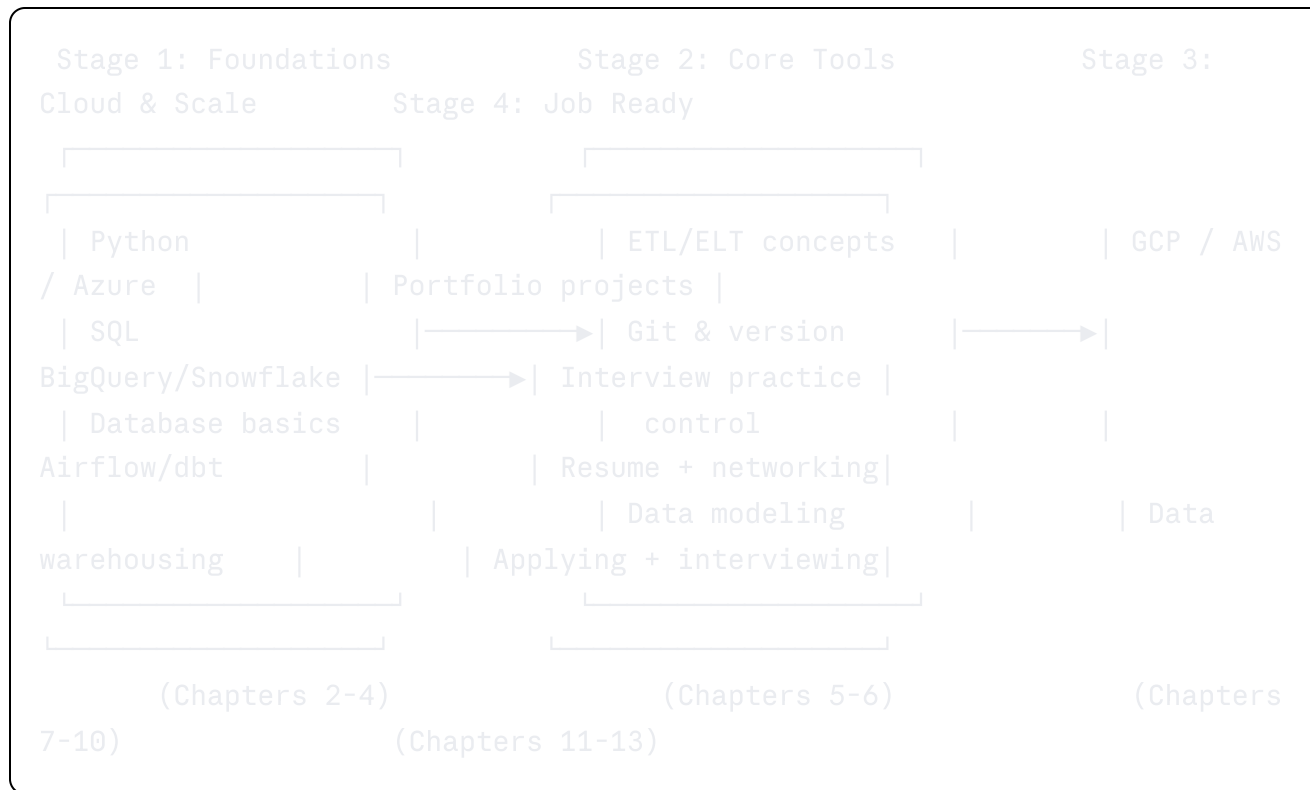
<div style="page-break-after: always;"></div>

# CHAPTER 13: Career Preparation

## 13.1 Learning Objectives

- Understand the typical roadmap from beginner to job-ready Data Engineer.
- Prepare for technical interviews with confidence.
- Build a portfolio that demonstrates real skill to employers.
- Know which certifications and resources are worth pursuing.

## 13.2 The Roadmap



This handbook, read chapter by chapter, mirrors exactly this progression.

## 13.3 Building a Portfolio

Employers want to see you can build something real, not just answer quiz questions.

Strong portfolio projects:

- **Are end-to-end**, like Chapter 11's project — from raw data all the way to a dashboard.
- **Use real (or realistic) messy data** — showcase your cleaning and validation skills, not just a "happy path."
- **Live on GitHub** with a clear README, showing your Git workflow (Chapter 5) in practice.

- **Use cloud tools** — even the free tiers of GCP/AWS/Azure — to demonstrate real-world relevance.

### Suggested portfolio project ideas:

1. A pipeline ingesting a public dataset (e.g., public transit data, weather data) into BigQuery, modeled into a star schema, visualized in a free BI tool.
2. An Airflow DAG orchestrating a multi-step pipeline with data quality tests (dbt or custom Python assertions).
3. A small "data platform" repo showing Python cleaning scripts, SQL models, tests, and documentation together.

## 13.4 Certifications

| Certification                                      | Provider     | Focus                                              |
|----------------------------------------------------|--------------|----------------------------------------------------|
| Google Cloud Professional Data Engineer            | Google Cloud | GCP data services, pipeline design, ML integration |
| AWS Certified Data Engineer - Associate            | AWS          | AWS data services (Glue, Redshift, Kinesis)        |
| Microsoft Certified: Azure Data Engineer Associate | Microsoft    | Azure data services (Data Factory, Synapse)        |

Certifications are useful signals, especially early in a career, but they supplement — not replace — a working portfolio and solid interview performance.

## 13.5 Learning Resources

### Books:

- *Fundamentals of Data Engineering* — Joe Reis & Matt Housley
- *Designing Data-Intensive Applications* — Martin Kleppmann
- *The Data Warehouse Toolkit* — Ralph Kimball
- *SQL Performance Explained* — Markus Winand

### **Communities:**

- Locally Optimistic (Data Engineering/Analytics Slack community)
- r/dataengineering (Reddit)
- dbt Community Slack

### **Open Source Projects Worth Studying:**

- Apache Airflow ([github.com/apache/airflow](https://github.com/apache/airflow))
- dbt-core ([github.com/dbt-labs/dbt-core](https://github.com/dbt-labs/dbt-core))
- Apache Parquet / Arrow ([github.com/apache/arrow](https://github.com/apache/arrow))

Studying real, well-engineered open-source code teaches patterns and conventions far beyond what any single course can.

## **13.6 Interview Preparation**

Data Engineering interviews typically test four areas:

| Area                   | What's Assessed                           | How to Prepare                                                                             |
|------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------|
| SQL                    | Query writing under time pressure         | Practice on platforms like LeetCode (Database section), StrataScratch, Mode's SQL tutorial |
| Python/coding          | Data manipulation, algorithms             | Practice <code>pandas</code> transformations and general coding problems                   |
| System/pipeline design | Designing a pipeline for a given scenario | Practice explaining trade-offs (ETL vs. ELT, batch vs. streaming, schema design) out loud  |
| Behavioral             | Communication, past project ownership     | Prepare STAR-format stories about real (or portfolio) projects                             |

### Sample Interview Questions (Consolidated)

1. Explain the difference between a Data Engineer, Data Analyst, and Data Scientist.
2. Write a SQL query to find the second-highest salary in an `employees` table.
3. What is the difference between `INNER JOIN` and `LEFT JOIN`?
4. Explain ETL vs. ELT and when you'd choose each.
5. What is a star schema, and why is it used in data warehouses?
6. How would you design a pipeline to process 1 billion events per day?
7. What is partitioning, and how does it help query performance?
8. Explain ACID and give an example of why atomicity matters.
9. How do you ensure a pipeline is idempotent?
10. Describe a data pipeline you've built or would build, end to end.

## 13.7 Resume Tips for Junior Data Engineers

- Lead with concrete, quantified outcomes: *"Built a pipeline ingesting 2M+ daily events into BigQuery, reducing report generation time from 3 hours to 10 minutes,"* not just *"Worked with data pipelines."*
- List specific tools by name (Python, SQL, BigQuery, Airflow, dbt, Git) — recruiters and ATS systems scan for these.
- Link to your GitHub portfolio prominently.
- Keep it to one page as a junior candidate; depth of 2-3 strong projects beats a long list of shallow ones.

## 13.8 Mini Exercise

Write your own two-sentence "elevator pitch" answer to: "What does a Data Engineer do, and why did you want to become one?"

## 13.9 Challenge Exercise

Using the roadmap in section 13.2, create a personal 90-day study plan that references specific chapters of this handbook and at least one portfolio project milestone per month.

## 13.10 Common Mistakes

⚠ **Common Mistake:** Applying only to jobs titled exactly "Data Engineer." Related titles — "Analytics Engineer," "ETL Developer," "Data Platform Engineer" — often involve very similar (or identical) work.

⚠ **Common Mistake:** Building portfolio projects only with perfectly clean sample datasets. Employers want to see you can handle messy, real-world data — intentionally include imperfect data in your portfolio work.

⚠ **Common Mistake:** Memorizing answers instead of understanding trade-offs. Interviewers probe "why," not just "what" — be ready to explain *why* you chose Parquet over CSV, or ELT over ETL, in your own projects.

### 13.11 Best Practices

- Build 2-3 deep, end-to-end portfolio projects rather than many shallow ones.
- Practice explaining your projects out loud — interview performance is as much about communication as technical correctness.
- Keep learning after landing the job — Data Engineering tools evolve constantly; the fundamentals in this handbook will outlast any specific tool's popularity.
- Contribute to open source or write technical blog posts about what you learn — this builds both skill and visibility.

### 13.12 Interview Questions (Career-Focused)

1. **Q: Why do you want to work in Data Engineering specifically, rather than Software Engineering or Data Science?** A: A strong answer connects to genuine interest in building reliable systems and infrastructure that enable others (analysts, scientists, the business) to do their best work — grounded in specific experience or projects, not a generic answer.
2. **Q: Tell me about a time a pipeline you built failed. What did you do?** A: A strong answer follows a clear structure (situation, what broke, how it was diagnosed, how it was fixed, what was changed to prevent recurrence — e.g., adding monitoring or a data quality test).
3. **Q: How do you stay current in a field that changes as fast as Data Engineering?** A: Mentioning specific habits — following release notes for tools you use, reading engineering blogs (Netflix, Airbnb, Uber often publish detailed data engineering posts), and building small side projects with new tools — demonstrates genuine ongoing engagement.

### 13.13 Review Questions

1. What are the four stages of the roadmap from beginner to job-ready?
2. Why do certifications supplement rather than replace a portfolio?
3. Name three related job titles a Data Engineering skillset applies to besides "Data Engineer."
4. What four areas do most Data Engineering interviews test?

### 13.14 Chapter Summary

Becoming a job-ready Data Engineer follows a clear path: foundations (Python, SQL, databases), core tools (Git, ETL/ELT), cloud and scale (GCP, BigQuery, warehousing), and finally job readiness (portfolio, interview practice, applying). A strong portfolio with real, messy data and clear documentation, combined with genuine understanding of trade-offs (not memorized answers), is what separates successful candidates. The field rewards continuous learning — the specific tools will change, but the fundamentals in this handbook will remain relevant throughout your career.

### 13.15 Further Reading

- Google Cloud Professional Data Engineer certification guide ([cloud.google.com/certification/data-engineer](https://cloud.google.com/certification/data-engineer))
- Engineering blogs: Netflix Tech Blog, Airbnb Engineering, Uber Engineering
- [r/dataengineering "getting started" wiki](https://www.reddit.com/r/dataengineering/wiki/getting_started/)

---

<div style="page-break-after: always;"></div>

## Closing Notes

Congratulations on completing *The Complete Data Engineering Fundamentals*

*Handbook*. You have now covered the same foundational ground that Data Engineers at Netflix, Amazon, Spotify, and Google build on daily: programming, SQL, database design, version control, ETL/ELT, data warehousing, data formats, cloud infrastructure, BigQuery, a complete end-to-end project, professional best practices, and career readiness.

Data Engineering is a field where fundamentals endure even as specific tools rise and fall in popularity. The habits this handbook emphasized throughout — validating data before trusting it, preserving raw data, writing idempotent and tested pipelines, understanding trade-offs rather than memorizing answers — will serve you regardless of which company, cloud provider, or tool you use next.

**Your next steps:**

1. Revisit Chapter 11 and rebuild the end-to-end project yourself, from scratch, without looking at the code.
2. Set up a free-tier Google Cloud account and load a real public dataset into BigQuery.
3. Start your GitHub portfolio today — even one solid project beats a long list of tutorials followed.
4. Review the Interview Questions across all thirteen chapters as a single study sheet before any interview.

Welcome to the field. Build reliably, document clearly, and never stop asking *why* a pipeline is designed the way it is.